riverrun

The Anonymity Layer for Solana, Post-Quantum

Hide *who* acted, not just how much you moved.
One secret, a different identity in every app, and nothing a quantum computer can break.

In plain words

read this first: no math, no jargon

Our goal is to empower people and to make knowledge accessible.

Privacy should not be a luxury for the technical and the wealthy, and a whitepaper should not be a wall that only cryptographers can climb. riverrun puts control of your own exposure back in your hands, and this paper is written so that anyone can understand it. Making a powerful thing understandable is part of the work, not an afterthought.

> **A note on method.** A whitepaper should be readable by a normal person before it is readable by a cryptographer. So this one opens with the whole idea in plain words. Everything said here is made precise later in the paper, and each plain-words idea below carries a link to the exact section that proves it, so you can jump from any simple claim straight to its technical version. If the simple version and the technical version ever disagree, that is a bug in the paper, not a simplification. We think every tech whitepaper should be written this way, and we offer this as a small contribution to how they are written from here on.

The problem. A blockchain is like a wall of glass. Everything you do with money on it stays there, on display, forever. Your name is not written on it, but your behavior is: what you buy, how much, when, and in what order. Outside the glass, companies you never hired press their faces to it and write all of this down. From the pattern alone they can work out who you are, and guess what you will do next. On a blockchain, you are an open book.

Most privacy tools try to hide your *money*, the amounts and the balances. riverrun hides something different, and we think more important. It hides *you*: which person, out of a crowd, actually did a thing.

What riverrun does, in three pictures.

1. It hides you in a crowd of look-alikes. Imagine you want to do something without being watched, so you do it at the same moment as a crowd of people who all look exactly like you. A snoop sees the crowd but cannot tell which one was you. That is the whole trick. riverrun builds that crowd around your action, and cuts the thread that ties the action back to you. *(The precise version is the pool, §6 and §8.)*

2. It gives you a different face at every door. Today, "sign in with Google" means one identity follows you everywhere and Google sees all of it. riverrun is the opposite. From one key you get a different, unconnectable identity at every door: one in a vote, one in an app, one in an airdrop. Nobody can put the pieces back together into you. And yet each door can still make sure you go through only once: one vote, one claim. You are invisible without becoming a ghost who acts a thousand times. *(The precise version is riverrun ID, §5.)*

3. It cannot be broken later, not even by a computer that does not exist yet. A blockchain keeps everything forever, so a patient attacker can copy your data today and wait for a future, far more powerful computer to crack it. Most privacy tools use locks that such a computer will open. riverrun uses a different kind of lock, built only from a one-way scramble that even that future machine cannot undo. What you hide today stays hidden, for good. *(The precise version is §4.1 and §5.1.)*

And one more thing: it does not just promise, it checks. Every privacy tool tells you "you are hidden among 30 people." riverrun is the one that actually goes and counts the real crowd. Often the real crowd is far smaller than advertised, and sometimes it is just you, standing alone. riverrun measures that out loud, on the real network, so you know your true privacy before you rely on it. It even turns the same measuring tool on itself. *(The precise version is the ruler, §10.)*

Who this is for. Anyone who does not want to be an open book on a blockchain. A trader whose strategy gets copied the moment they move. A large holder who gets front-run. An ordinary person who does not want a stranger profiling their salary or their savings. And, going forward, any app that needs "anonymous, but one action per person": private voting, fair airdrops, reputation you can carry without a trail.

What is honest to say it cannot do yet. riverrun is real, tested, and open. The measuring tool works on the live network today. The privacy engine works and has been checked, including by attacking our own code and fixing what we found. But the piece that would let the blockchain itself verify everything, with no trusted helper, is built and proven in a test environment, not yet live on the main network. So do not guard real money or a real identity with the engine yet. We say exactly where each piece stands all through the paper, because a privacy tool that will not be honest about its own limits is not one you should trust with your privacy. *(Stated precisely in §13, the self-audit in §13.1, and the full list in §28.)*

That is riverrun, in plain words. The rest of this paper is only these few ideas, made precise and made real.

riverrun, past Eve and Adam's, from swerve of shore to bend of bay,
brings us by a commodius vicus of recirculation.
James Joyce, *Finnegans Wake*, first line

Abstract. *Solana has no Semaphore. It has no native way to carry one identity across apps without linking them, and no anonymous-but-accountable primitive at all. This paper describes* RIVERRUN*, the anonymity layer that fills that gap, and it is post-quantum. Its core is* ***riverrun ID****: from a single secret, a user gets a different, unlinkable identity in every app, while each app can still enforce one action per person. Because it is built from hashes and nothing else, that unlinkability is everlasting. A future quantum computer cannot undo it, which the deployed elliptic-curve pseudonym standards cannot say. On top of the identity layer sits a* ***pool*** *that hides a behavior instead of a coin, proven by a transparent post-quantum STARK whose on-chain verification we show inside the Solana VM at* 159,849 *compute units. And a* ***ruler*** *keeps the whole thing honest: it measures the real anonymity a set delivers, and against a live pool on Solana mainnet a crowd that advertised* 30 *was really worth* 6.5*, with one member alone. Every claim of the form "X is bound" has a test we deleted the constraint to watch fail. A self-audit closed three real findings, each with a test. Where the work is unfinished, an unaudited circuit, on-chain verification proven in a VM but not yet on mainnet, we say so plainly. Every number here comes from one command.*

Manuel Guilherme Galmanus • Independent Researcher • Preprint, July 2026

Privacy is not secrecy. Secrecy is hiding what you did from everyone. Privacy is the power to choose who sees it. On a public ledger you have neither by default. Every action you take is pinned to a board that never forgets, and firms you never hired read it for a living. No exchange, no analytics company, no protocol will hand you behavioral privacy out of kindness. If we want it, we build it, and we build it to outlast the machine coming to break it. Hash-only, no trusted setup, no one to ask permission from. The cypherpunks wrote code to defend privacy. We write code, and we add the one thing the old manifestos left to faith. We do not ask you to trust that the privacy is real. You can check it yourself.

Contents

1 The problem, in plain words

Picture a town where every letter anyone ever mailed is pinned to a board in the square, forever. Not the contents, those are sealed, but the envelopes: who sent what to whom, when, and how heavy it was. You would learn a great deal about that town without opening a single letter. Who pays whom. Who is suddenly buying. Who moves money at three in the morning. Who copies whose move an hour later.

A public blockchain is that board. People think that a wallet address, being a random string and not a name, keeps them private. The opposite is closer to the truth. The address is a fixed handle, and everything done under it piles up into a pattern. How much you move, which apps you touch, in what order. Modern chain-analysis uses that pattern to link your wallets, attach a name, and increasingly to predict and front-run what you will do next. These tools get better every month, because the data is permanent and public and the models feeding on it are hungry.

Most privacy work assumes the problem is the money. Hide the amounts, hide the balances. That view is incomplete. There are two kinds of privacy here, and they are not the same. One hides *how much*. The other hides *who*, meaning which person, out of a crowd, did a public thing. Almost all the work so far is on the first: confidential transfers, shielded balances, mixers that break the link between a deposit and a withdrawal of value. This paper is about the second. Amounts are not hidden here and are not the point. What we cut is the link between an *actor* and an *action*.

> Here is the whole idea in one sentence, and the rest of the paper is only this sentence made precise and made real. **You disappear into a crowd of people who all look like they could have done the thing you did.**

> **On the name.** *Finnegans Wake* (James Joyce, 1939) is a novel written as a circle. Its last sentence breaks off mid-phrase and runs straight into its first word, "riverrun." We did not pick the name for the poetry. One idea from the book is load-bearing and shows up later as a number: a circle has no beginning, and a funding trail that loops back on itself has no origin to trace (§10). That is the strongest defense the ruler finds.

2 What privacy is, and why behavioral privacy is its hardest case

A privacy tool that cannot say what privacy *is* will build the wrong thing. So before the cryptography, the concept. This section argues four claims that the rest of the paper depends on: privacy is not secrecy, privacy is a right and not a mere preference, anonymity is a distinct and collective good, and behavioral privacy, hiding *who* rather than *how much*, is the hardest case of all and the one most worth solving on a public ledger.

2.1 Privacy is not secrecy, and not the same as anonymity

The three words are used as if interchangeable, and they are not. **Secrecy** is hiding a fact from everyone. **Privacy**, in the definition we take from the cypherpunk tradition and which matches

ordinary intuition, is the *power to selectively reveal yourself to the world*: to choose who learns what, and when. A private matter is not one nobody may know. It is one you decide who knows. You are not keeping your salary secret when you tell your bank and hide it from a stranger on the street. You are exercising privacy, which is control over an audience.

Anonymity is a third thing again. It is unlinkability between an actor and an act. A message is anonymous not when its contents are hidden, they may be fully public, but when no one can tie it to its author. This is the good riverrun defends. The amounts are not hidden, the actions are not hidden, the fact that *someone* acted is not hidden. What is severed is the link from the act back to the person.

> A worked distinction. A sealed envelope is **secrecy**: the letter inside is hidden from all. A letter you show your lawyer but not your neighbor is **privacy**: you choose the audience. A signed-but-unaddressed leaflet dropped in a crowd, whose author no one can name, is **anonymity**: the content is open, the author is not. Most blockchain privacy work builds the first, confidential amounts. riverrun builds the third, and offers the second, selective revelation, as a set of user-controlled powers (§5).

The distinction is old. Warren and Brandeis founded the modern legal concept on "the right to be let alone" [22]. Westin reframed it as informational self-determination, the claim of individuals to determine for themselves when and how information about them is communicated [23]. Gavison analyzed it as a measure of the access others have to you, along the axes of secrecy, anonymity, and solitude [24]. Across these, privacy is control, not concealment, which is why a system that only conceals amounts leaves the person exposed.

2.2 Privacy is a right, because its absence is a chilling force

Is privacy a preference, a luxury one may forgo, or a right, a precondition for other goods? The argument for the second is not sentimental. It is structural, and it runs through the **panopticon**.

Bentham designed the panopticon as a prison in which a single guard might watch any cell, and no prisoner could tell when they were watched [25]. Foucault saw that the genius of the design was not the watching but the *uncertainty* of it: not knowing when you are observed, you behave at all times as if you are [26]. The watcher's power becomes automatic and internalized. The prisoner guards themselves.

A public ledger is a panopticon with the uncertainty removed and replaced by something worse, a certainty that runs backward in time. Every action is not merely watchable, it is recorded, permanently, and readable by anyone, forever. The chilling effect that Foucault described as a tendency becomes, on-chain, a fact. A person who knows their every payment is public and permanent does not transact freely. They avoid the controversial donation, the sensitive purchase, the association that could be held against them in a future whose norms they cannot predict. This is the *disanalogy that makes the ledger worse than Bentham's prison*: the panopticon's inmate is watched only in the present, and forgotten when released, while the ledger's subject is watched in a present that never ends, by observers who have not yet been born, under laws not yet written.

> The chilling-effect argument is empirical, and honesty requires the state of the evidence. Studies of surveillance and self-censorship (for instance, measured drops in sensitive search traffic after mass-surveillance disclosures) support the direction of the effect, but its magnitude on-chain is not something we have measured, and we do not claim a number. What we claim is narrower and defensible: a permanent public record of behavior creates a structural incentive toward self-censorship, and removing the link from act to actor removes that incentive. Whether any given user feels chilled is theirs to say. That the architecture chills is a property of the architecture.

2.3 "Nothing to hide," and why it fails

The standard objection to privacy is that an honest person has nothing to hide, so only wrongdoers want it. The argument fails on three counts, and naming them sharpens what riverrun defends. First, it **mistakes the good**. Privacy is not the concealment of wrongdoing but control over an audience (§2); the person who closes the bathroom door has nothing to hide and everything to keep private, and the distinction is not about guilt. Second, it **assumes a fixed and benevolent observer**. On a permanent ledger the observer is not one authority today but every authority forever, under norms that change: an act legal and ordinary now may be reread by a future regime, employer, or insurer, and the record does not expire. The person with nothing to hide today cannot know they have nothing to hide in 2050, which is the timescale a permanent record actually spans. Third, it **ignores aggregation**. Solove's analysis [38] shows that harm accrues not from any single disclosure but from the assembly of many innocuous facts into a profile that predicts, sorts, and prices a person, which is precisely what behavioral chain-analysis does. The response to "nothing to hide" is that privacy protects against the aggregate future use of the present record, and no individual act reveals whether that protection was needed.

2.4 Contextual integrity: privacy as appropriate flow

Nissenbaum's framework of **contextual integrity** [39] gives the sharpest account of what riverrun ID implements. Privacy, on this view, is not secrecy and not a binary of public-versus-private, but the preservation of *appropriate information flows*: each social context, a bank, a vote, a clinic, a marketplace, carries norms about what information flows to whom, and a violation is not disclosure as such but disclosure that breaks the context's norm. Your pharmacist knowing your prescription is appropriate; your employer knowing it is not, even though both are "someone knowing."

A public ledger destroys contextual integrity by collapsing all contexts into one: every flow is visible to every observer, so the norms that separate a vote from a purchase from a donation cannot hold. riverrun ID is contextual integrity made cryptographic. The **shape** power gives each context a distinct, unlinkable identity, so information that flows within one context does not leak across to another, and the **link** power lets the holder, and only the holder, join two contexts when the norm permits it. The design target is not to hide everything from everyone, which is secrecy and which no useful system provides, but to restore the boundaries between contexts that the ledger erased. This is why the identity layer is central and the pool is one application: the deeper good is appropriate flow, and hiding a single behavior is one instance of it.

2.5 Surveillance is an economy, not an accident

The chilling effect would matter less if surveillance were rare. It is not, because it is profitable. Zuboff's account of **surveillance capitalism** [40] describes an economic order in which human behavior is the raw material: it is captured, rendered into data, and traded as prediction, and the incentive to capture more is structural, not malicious. A permanent public ledger is the ideal input to this economy, a stream of behavioral data that is free to collect, impossible to delete, and attributable at increasing accuracy as the models improve. The market for chain-analysis, firms that de-anonymize on-chain behavior for exchanges, regulators, and traders, is the concrete instance, and it grows because the data grows and the models feeding on it get better.

This reframes riverrun's purpose in economic terms. The point is not to defeat a particular adversary but to *raise the cost* of behavioral surveillance from near zero toward something, and to give the individual a tool whose protection does not depend on the goodwill of the firms profiting from its absence. The cypherpunk premise of §3, that no large organization grants privacy out of beneficence, is not cynicism but a correct reading of the incentive structure: the parties who could grant behavioral privacy are the parties who profit from selling its absence, so the only durable source of it is a tool the individual runs for themselves.

2.6 Anonymity is a collective good, not only a private one

Here is the fact that makes anonymity unlike most goods, and that shapes every design decision in this paper. **Your anonymity is made of other people.** A secret you hold alone, a password, is yours entirely. But you cannot be anonymous alone. To be unlinkable to your act, there must be others who could equally have done it. The *anonymity set* is that crowd, and it is a shared resource. When you join a privacy pool you consume some of its anonymity and also contribute to everyone else's. When you leave, or when you stand out, you diminish theirs.

This has three consequences that run through the paper. First, anonymity is *non-excludable and rivalrous in a specific way*, closer to a commons than to private property, which means it can be depleted by misuse (a member who acts a thousand times, §6) and must be protected by rules that bind everyone (the nullifier, one act per member). Second, the *quality* of the set is not given by its size, because the members are not interchangeable if their funding origins differ, which is the entire subject of the ruler (§10). Third, forming a good set is a coordination problem, not an individual one, which is why riverrun includes an agent that forms crowds on purpose (§11) rather than hoping a good crowd appears.

> The structural isomorphism, stated and then bounded. Anonymity behaves like herd immunity in epidemiology: an individual's protection depends on enough others also participating, and free-riders who take protection without contributing weaken the whole. *Disanalogy:* herd immunity protects even non-participants once a threshold is crossed, whereas an anonymity set protects only its members, and a non-member gains nothing from the crowd's existence. The commons is real, the externality runs only inward.

2.7 Behavioral privacy is the hard case, and the neglected one

There are, we said in §2 above and will not repeat, two privacies on a ledger: *how much* and *who*. The field has poured its effort into the first. Confidential transfers, shielded balances, and value-mixers hide amounts, and they are mature and valuable. Behavioral privacy, hiding which person out of a crowd performed a public act, has been comparatively neglected. Three reasons, and each is a reason it is the harder problem.

First, **behavior is higher-dimensional than value**. An amount is one number. A behavior is a pattern: timing, sequence, counterparties, sizing, the choice of venue. Chain-analysis clusters on that pattern, and hiding it means hiding a shape, not a scalar. Second, **the leak is often outside the protocol**. Even a perfect in-protocol mix leaves the funding graph visible, and the funding graph re-identifies the actor, which is why the ruler exists and why we treat provenance as a first-class adversary channel rather than an afterthought (§10). Third, **behavioral privacy is what predictive adversaries want**. An adversary who learns your balance has learned a fact. An adversary who learns your behavioral fingerprint can *predict and front-run* you, which is a strictly greater harm, and it is the harm that grows as the models feeding on the permanent public record improve.

The claim of this paper is that behavioral privacy is both the harder problem and the more valuable one, and that Solana, in particular, has no primitive for it at all. That absence is the opportunity riverrun addresses. The philosophy is not decoration. It sets the target precisely: not to hide the money, which others do well, but to sever the actor from the act, permanently, in a way a future machine cannot reverse, while being honest at every step about how large the remaining crowd really is.

3 The cypherpunk line, and where riverrun turns it

riverrun stands in a tradition, and naming it precisely is both an intellectual debt and a way to state what is new. The tradition is the cypherpunk one: the claim that privacy in the electronic age is not granted by institutions but built by individuals writing code. This section traces that line, from its origin in the cryptography of the 1980s to its collision with the law in the 2020s, and then states the one turn riverrun tries to add to it.

3.1 Chaum: privacy is a cryptographic problem, not a policy one

The line begins with David Chaum. In 1981 he described the **mix network**, a way to send mail so that no observer, not even the relays themselves, could link a sender to a receiver [27]. The mechanism, wrapping a message in layers of encryption that each relay peels and shuffles, is the direct ancestor of every anonymity network since, including Tor, and its core idea, that unlinkability can be a mathematical property rather than a promise, is the seed of everything here. In 1985 Chaum went further, arguing in "Security without Identification" that a society could run its transactions, credentials, and payments without a central register of who did what, and that the alternative, a "dossier society" of total records, was a choice, not a necessity [28]. His DigiCash implemented anonymous electronic money a decade before Bitcoin. Chaum's contribution to riverrun is foundational and specific: the idea that anonymity is a construction, that a nullifier-like token can prevent double-spending without naming the spender, and that the goal is a system where identification is the exception a user chooses, not the default the system imposes.

3.2 May and Hughes: the manifestos

If Chaum supplied the mathematics, Tim May and Eric Hughes supplied the politics. May's *Crypto Anarchist Manifesto* (1988) predicted that cryptography would let people interact and transact without revealing identity, and that this would shift power away from institutions whose leverage rested on their monopoly over records [29]. Hughes' *A Cypherpunk's Manifesto* (1993) is the more disciplined document, and the one riverrun takes most directly [30]. Three of its lines are load-bearing here.

> *"Privacy is necessary for an open society in the electronic age. Privacy is not secrecy. A private matter is something one doesn't want the whole world to know, but a secret matter is something one doesn't want anybody to know. Privacy is the power to selectively reveal oneself to the world."*

That is the definition we adopted in §2, and it is exactly what riverrun ID's **link** power makes cryptographic: the holder chooses which of their identities to reveal as one, to whom, for which contexts (§5).

> *"We cannot expect governments, corporations, or other large, faceless organizations to grant us privacy out of their beneficence. . . . We must defend our own privacy if we expect to have any."*

That is the reason riverrun is non-custodial and transparent, with no trusted party who could grant or revoke the privacy, and no ceremony whose secret could betray it.

> *"Cypherpunks write code. We know that someone has to write software to defend privacy, and . . . we're going to write it."*

That is the reason this paper ships with an implementation, tests, and reproducible numbers rather than a specification alone.

3.3 The crypto wars, and the lesson that code is speech

The manifestos were written under siege. Through the 1990s the United States classified strong cryptography as a munition and sought to restrict its export and to mandate key escrow through the Clipper chip, a hardware backdoor. The cypherpunks fought this in code and in court, and the eventual recognition that source code is protected expression established the principle that one cannot be forbidden from writing or publishing a cryptographic tool [31]. The lesson riverrun inherits is not triumphalism. It is that the defense of privacy is adversarial and political, that the pressure to insert a trusted party or a backdoor is constant, and that the only durable answer is an architecture in which there is no trusted party to coerce. A committee is a compromise, and §13 says so plainly and treats its removal as the roadmap.

3.4 PGP: privacy put in ordinary hands

The manifestos would have stayed manifestos without a tool a person could actually run. Phil Zimmermann's PGP (1991) was that tool, strong encryption for email, released freely and spread worldwide before the export regime could contain it [41]. Its significance for riverrun is twofold. First, it demonstrated the cypherpunk thesis in practice: privacy was not granted by any institution

but seized by publishing code that individuals could run themselves, and once published it could not be recalled. Second, it set the standard that a privacy tool must be usable by non-experts, because a tool only cryptographers can operate protects only cryptographers. That standard is the reason this paper opens with an ELI5 rather than an abstract, and the reason the design goal (§2) is a primitive an ordinary person invokes with one key, not a protocol only the technical can wield. Privacy that is a luxury for the skilled is not the open society Hughes wrote about.

3.5 The pre-history of on-chain money: b-money and bit gold

Between the cypherpunk mailing list and Bitcoin lie two proposals that riverrun inherits structurally. Wei Dai's **b-money** (1998) sketched a currency maintained by a distributed set of record-keepers rather than a bank, with pseudonymous participants and proof-of-work to make identities costly [42]. Nick Szabo's **bit gold** (around 1998) proposed unforgeable costly bits chained together, anticipating the hash-linked structure of a blockchain [43]. Both are ancestors of Bitcoin, and both carry the lesson riverrun takes as a constraint: a system meant to resist coercion must have no central party whose compromise breaks it, because any such party is a point the adversary of §3 will pressure. It is the same reason riverrun's soundness rests on a hash and not a ceremony, its accountability on self-revealing algebra (§16) and not a trusted opener, and its committee, where one still exists, on a threshold of distinct signers rather than a single authority, with its removal named as the goal.

3.6 On-chain: pseudonymity, Zcash, Tornado, and the law

Bitcoin gave the tradition a public ledger and, with it, a lesson in the gap between pseudonymity and anonymity. A Bitcoin address is a pseudonym, not a mask, and a decade of chain-analysis has shown how thoroughly pseudonyms de-anonymize under a permanent public record. Zcash answered with shielded transactions, hiding amounts and parties behind zero-knowledge proofs, the most complete on-chain privacy deployed at scale. Tornado Cash answered for Ethereum with a non-custodial value-mixer, a Merkle set of deposits and a zero-knowledge withdrawal, whose shape riverrun borrows and whose payload it changes from a coin to a behavior (§6).

Then came the law, and it is the part of the lineage a responsible paper cannot skip. In 2022 the United States sanctioned Tornado Cash itself, the first time a piece of autonomous code, rather than a person or entity, was placed on a sanctions list. A developer, Alexey Pertsev, was convicted in the Netherlands in 2024, and further prosecutions followed. The precedent is sobering and directly shapes riverrun's design and its business model: a tool that takes a fee to anonymize *value transfer* exposes its operators to money-transmission and sanctions liability, whereas a tool that hides *who acted* and takes no custody of value sits differently, and an identity and measurement layer sits differently still. We state this not as legal advice, which we are not qualified to give, but as a design constraint we took seriously: riverrun does not custody funds, does not mix value for a fee, and treats the honest disclosure of its own limits as a feature, not a vulnerability.

3.7 The turn riverrun tries to add: from promise to proof

Every generation in this line added one thing. Chaum added that anonymity can be constructed. May and Hughes added that it must be self-defended. Zcash and Tornado added that it can run on a public ledger. What the tradition never fully supplied is a way for the user to *know*, rather than

trust, how much privacy a given tool actually gives them. A mixer tells you that you are hidden among your fellow depositors. It does not tell you that, once the funding graph is read, your real crowd is a fraction of that, and sometimes just you. The cypherpunks wrote code to defend privacy. riverrun's small addition is to write code that also *measures* whether the defense worked, on the real network, and reports the answer honestly, including when the answer is uncomfortable. That is the ruler (§10), and it is why the credo on the first page ends not with "trust us" but with "check."

> Standing in a tradition is not the same as extending it, and we should not overclaim the turn. Measuring anonymity is itself not new: the metric we use is from 2002 (§14), and measuring advertised-versus-real anonymity is an established research programme on Ethereum. What is uncommon is doing it as a shipped, user-facing tool, on Solana, on live data, and wiring the same honesty into the identity layer and the on-chain claims. The contribution is integration and discipline, not a new primitive, and we say which parts are which throughout.

4 What riverrun is

Ethereum has Semaphore. From one secret, a user gets an unlinkable identity in every context, and each context can still stop them from acting twice. A whole category of apps is built on it: anonymous voting, sybil-resistant airdrops, private reputation. Solana has no equivalent. RIVERRUN is that missing layer, and it is post-quantum. It is three things that fit together.

1. **An identity layer**, *riverrun ID* (§5). The core. One secret becomes a different, unlinkable identity in every app, while each app can still enforce one action per person. Because it is hash-based, the unlinkability is everlasting, and a quantum computer cannot undo it. The primitive is built and tested. The on-chain verifier is the road ahead.
2. **A pool** (§6–§9). One application of the identity layer, and the research core. A member commits an intent and later carries it out, with no observer able to say which member did it, proven by a transparent post-quantum STARK.
3. **A ruler** (§10). The thing that keeps the rest honest. It measures how much anonymity a set *really* gives, not the number it advertises. It reads only public chain data, works on sets it has never seen, and runs on Solana mainnet today.

The through-line is one design choice made everywhere. **Build from a single hash function, and nothing else.** No elliptic curves, no pairings, no trusted setup ceremony. That is what makes the whole thing *post-quantum*. A large quantum computer breaks the curve-based cryptography under most of today's privacy tools, but it does not break a hash. Section 4.1 says exactly why that matters now and not in ten years.

4.1 Why a tool like this must be post-quantum, not *may*, *must*

The usual objection is that a quantum computer big enough to break today's cryptography does not exist yet. True, and beside the point. The argument for post-quantum here is not about fear. It is an inequality, due to Mosca. Let X be how long a secret must stay secret, Y the time to migrate a system to quantum-safe cryptography, and Z the time until a cryptographically-relevant quantum

computer exists. If

$$X + Y > Z,$$

the machine arrives before the protection does, and everything recorded in the meantime is decrypted retroactively.

Now put a blockchain anonymity set into that inequality. The ledger is public and permanent. The link between an actor and their action, if it survives at all, survives forever, so $X = \infty$. No finite Y or Z satisfies the inequality. For a privacy tool whose data lives on a permanent public ledger, $X + Y > Z$ is not a risk to manage. It is a *certainty*, unless the primitive is *already* quantum-safe at the moment of writing. This is what **harvest-now-decrypt-later** means made precise. An adversary needs no quantum computer today, only a copy of the chain, which is free. The day the hardware exists, every curve-based guarantee ever written to that chain fails at once, including the ones written in 2026.

Disanalogy, stated. For ephemeral secrets, a TLS session key, a monthly-rotated password, X is small, and a non-post-quantum scheme is defensible for a few more years. That is exactly the reasoning that does not transfer here. On-chain anonymity has no expiry, so the comfort ephemeral data enjoys does not apply.

Curve-based privacy tools fail the test concretely. Shor's algorithm breaks the curve, and the recorded data is already in the adversary's hands. Hash-based tools pass it. The best known quantum attack on a hash is Grover's, which only halves the security level, and a 256-bit hash absorbs that with room to spare. RIVERRUN is hash-only by design, so a user who hides a behavior today is still hidden after quantum hardware exists, at no extra cost, because the property is structural, not bolted on. The standards bodies have already acted on this for data far less permanent than a ledger. NIST finalized its post-quantum standards (FIPS 203/204/205) in August 2024, and its hash-based signature, FIPS 205 (SLH-DSA), rests on the same conservative assumption RIVERRUN does. Read the other way: in 2026, a permanent-ledger privacy tool that is *not* post-quantum is shipping a guarantee it already knows will expire.

5 riverrun ID: one secret, seven powers

Start with the picture that started it. Take a jigsaw piece. It has a shape. Turn it over and the back has a different shape. Turn it to a new angle and it fits a different neighbor. *One piece, many shapes, each shape its own matching fit.* riverrun ID is that piece, in cryptography.

A user holds one **secret** s. A **context**, call it an angle θ, is any public label: an app, a DAO, an airdrop, a voting round. From the one secret and a context, the user derives an identity for *that* context and nothing else. Same secret, different angle, a different and unlinkable identity, each with its own matching fit. Seven powers come out of this, and every one is a hash plus, in one case, a little polynomial arithmetic, so all seven are post-quantum.

#	power	what the user can do
1	**shape**	be a different, unlinkable identity in every context
2	**fit**	act *once* per context, sybil-resistant
3	**turn**	prove "I am the same person across two rounds" in zero knowledge, revealing to no one *which* person
4	**link**	choose to reveal that two of your identities are one, to whom you pick, for the contexts you pick
5	**grant**	lend one context to an agent, bound to that agent, scoped to that context, never handing over your master secret
6	**rln**	rate-limit yourself: N actions stay anonymous, the N+1-th lets anyone unmask you
7	**credential**	carry an issuer's attribute ("verified", "over 18") and show it per context, without a trail

Put together, the user is **invisible by default** (1), **accountable where it matters** (2, 6), **continuous when they choose** (3), **linkable only on their own terms** (4), able to **lend a single context to an agent** (5), and able to **prove a fact about themselves with no trail** (7). One secret, full control of your own exposure, quantum-safe throughout.

> Two powers carry the weight. **shape** makes you unlinkable. Your identity in a DAO and your identity in an airdrop are two different hashes of the same secret, and no one can tell they belong to one person. **fit** makes you accountable. It is a token you can spend only once in a context, so "one vote per member" or "one claim per person" holds *without* anyone learning who you are. Invisible and one-per-person at the same time. That pair is the whole trick, and everything else is a refinement of it.

5.1 Everlasting unlinkability, and why it is the strong claim

"Post-quantum" undersells what riverrun ID has. The pseudonym schemes actually being standardized and deployed, BBS per-verifier pseudonyms, are unlinkable only *computationally*, under the discrete-log assumption. Their own literature states the consequence plainly: this "reduces perfect privacy to computational privacy, which can be retroactively broken when efficient quantum computers become available. Thus the simple construction should only be used when *everlasting* privacy is not required" [9]. That is the harvest-now-decrypt-later problem arriving from inside the anonymous-credential world.

riverrun ID's **shape** and **fit** are on the everlasting side by construction. They are hashes, with no discrete-log to break, so the link between a user's identities in two contexts cannot be recovered even by a future quantum adversary holding today's transcript. This is not an invention out of nowhere. Kazmi and Minwalla built the first secret-free, quantum-safe anonymous credential from a Merkle tree of commitments and a quantum-safe hash [8]. riverrun ID is of that class. The precise, defensible claim is therefore stronger than "a post-quantum Semaphore." It is that riverrun ID provides **everlasting per-context unlinkability**, a property the deployed pseudonym standard does not have, and which its own literature flags as the open requirement.

5.2 Erosion under repeated use, and how the identity rotates itself

There is an honest limit, and it is the interesting part. The identity's value is that you use it *repeatedly*, across many contexts and cycles. Cryptographic unlinkability hides the pseudonym, but a persistent quasi-identifier can still ride underneath every use. The clearest one is funding provenance, the very channel the ruler measures (§10): your funding origin is the same in every context, so an adversary can link your actions by the *origin* rather than the pseudonym, and then intersect. This is the classic intersection attack, and Danezis, co-author of the metric the ruler uses, warned that single-shot entropy is "very poor" at measuring repeated use [2].

So we measure it. For a persistent identity across n uses, the effective anonymity is the running intersection of the candidate sets, $k_{\text{eff}}^{(n)} = |U| \cdot 2^{-\sum_i \varepsilon_i}$, where ε_i is the leak of use i. The number decays as the same identity acts more. The closure is that when the accumulated leak crosses a floor, the defense is already a power: **turn** rotates the secret to a fresh lineage with an un-intersected candidate set. The metric does not just warn, it *fires* the rotation. An anonymous identity that measures its own erosion and rotates itself before it is named.

> **turn** resets the crypto lineage, not the physical quasi-identifier. If you rotate the secret but keep funding from the same rare origin, the origin-based intersection survives the rotation. So the two defenses compose: rotate the secret, and re-fund the next lineage from a common origin. Rotating alone buys less than it looks. The metric is computable (6 tests) and is a floor under a stated adversary, the same discipline as the ruler.

5.3 What it unlocks

Each of these is just a different context, a different angle on the same secret:

- **Anonymous DAO voting**: one vote per member, nobody learns who voted how.
- **Sybil-resistant airdrops**: one claim per person, unlinkable to their main wallet.
- **Portable, unlinkable reputation**: prove "verified member" across apps with no trail joining them.
- **Rate-limited anonymous posting**: act under a per-context cap, spam priced in your own de-anonymization.
- **Delegation to agents**: let a bot act as you in *one* app, without giving it the keys to the rest of your life.

> The seven-power primitive is built and tested in the core crate (48 tests), and the "turn" power is proven in zero knowledge over the same STARK the pool uses. What is *not* yet built is the in-circuit membership proof over a small field, its Solana verifier, and an audit. Those are integration work on vetted parts (Plonky3's Poseidon2, the M31 verifier class of §13), not new mathematics, but they are not done. This section is the product's north star, not a shipped claim.

6 The pool: commit an intent, act unlinkably

The pool is the first thing you can build on the identity layer, so it is where the research went deepest. Tornado Cash, the Ethereum mixer, breaks the link between putting money *in* and taking money *out*. You deposit one coin, and later, from a fresh address, you withdraw one coin, proving you are owed a withdrawal without revealing which deposit was yours. The pool is a crowd, and you hide in it.

RIVERRUN keeps the shape and changes what is hidden. Instead of a coin, the hidden thing is a *behavior*. Three steps.

Commit. A member publishes $\ell = \text{Rescue}(s, a)$ into a public set, where s is a secret only they know and a is the action they intend, "swap 1 SOL for USDC on venue V", encoded as a hash. Publishing ℓ announces that *somebody in this set intends action* a, and nothing about who. The set is a Merkle tree, and its root R is the public fingerprint of the whole crowd.

Execute. Later, ideally in a round where many members act at once, the member does a from a fresh identity, attaching a zero-knowledge proof of one sentence: *"I know a secret* s *whose commitment* $\text{Rescue}(s, a)$ *sits under the public root* R*, and my nullifier this round is* $n = \text{Rescue}(s, r)$*."* The action is public. The author is not.

Settle. The pool checks the proof, checks the nullifier n is fresh this round, spends it, and releases the action. The public record is $\{R, a, n\}$, with no link back to any commitment.

Two questions decide whether this is real privacy or theater, and both are about the nullifier. Why have one? Because without it a member could act a thousand times from a thousand fresh identities, and the "crowd" would be one person in a thousand masks. The nullifier is a token derived from the secret and the round, spendable once per round. It reveals nothing about s, and one member's nullifiers in different rounds cannot be tied together. So it enforces "one action per member per round" while never naming the member. This is a studied primitive. PLUME [7] formalizes exactly it.

The harder question: how does anyone know the revealed n came from the *same* secret that proved membership? If those two facts are proven separately, a member could prove membership honestly and then present a stranger's nullifier, spending someone else's turn, or hiding that they spent their own. The two halves have to be welded inside one proof. Getting that weld right, and *knowing* it is right, is the next section.

7 The proof: one object, three bindings

A zero-knowledge proof convinces you a statement is true while telling you nothing beyond its truth. Two families are used on blockchains, and the choice is a real fork.

Pairing-based SNARKs (Groth16) give tiny proofs, a few hundred bytes, that verify cheaply. Their price is a *trusted setup*: a one-time ceremony makes a secret proving key, and if that secret ever leaks, the whole system can be forged. The key is tens of megabytes that must be produced, trusted, and shipped to every prover. And the security rests on elliptic curves, which a quantum computer breaks.

STARKs are built from a hash function alone. No pairings, no ceremony, no per-circuit setup, and security rests on the hash, believed to survive a quantum adversary. The price is proof size, kilobytes, not bytes. RIVERRUN takes this side on purpose, and §13 measures exactly what it costs.

What a STARK is, in one paragraph. Write your computation as a big table. Rows are time-steps, columns are registers, arranged so "it ran correctly" becomes "every pair of neighboring rows obeys a fixed rule." Turn each column into a polynomial. The rules become equations those polynomials must satisfy. The prover commits to the polynomials, and the verifier challenges them at random points. A cheat would need a *wrong* low-degree polynomial that agrees with a right one almost everywhere, which is impossible unless they are equal. The verifier never sees the answers, only a few random cells and a proof that all the cells are consistent, which is why the witness stays hidden.

7.1 What the circuit proves

RIVERRUN uses a STARK over a Rescue-Prime [14] Merkle tree, a hash chosen because it is cheap to write as polynomial constraints. The proof's public inputs are the four values $\{R, n, r, a\}$: root, nullifier, round, action. Its private inputs, the witness, are the secret s, the leaf's position, and the Merkle path. One secret must satisfy all three of these at once:

1. **Membership.** Rescue(s, a) is a leaf under R, via the private path.
2. **Nullifier binding.** $n =$ Rescue(s, r): the revealed nullifier is this member's, this round.
3. **Action binding.** The leaf under R commits to the public action a: the member does the intent they registered, not another.

Because all three read the same private s, no member can mix and match. That is the weld.

7.2 Knowing the weld holds

A privacy proof that is subtly wrong is worse than no proof, because it advertises a guarantee it does not deliver. So the interesting question is not "did we write the constraints" but "do they bite." The method used throughout RIVERRUN is *mutation testing*. For every claim "X is bound" there is a test, and we delete the constraint meant to enforce X and confirm the test then fails. If deleting it changes nothing, the test was passing for some other reason and is worthless.

This is not a formality. Three tests written during development were exactly that worthless, and mutation testing is how they were caught. The action-binding tests, for one, passed even with the binding deleted, because the action also feeds the proof's transcript, so a mismatched public input was rejected for an unrelated reason. The real attack hashes the committed action into the leaf, so the path still resolves, while *announcing* a different one. Delete the constraint and that forged proof verifies. The useful test performs that attack. Two other tests, a nullifier check and, in the ricorso, a whole migration check, had the same disease and the same cure.

None of this makes the circuit audited. It is hand-rolled, and a passing negative test is necessary, not sufficient, for soundness. An independent review is on the roadmap. What mutation testing removes is one specific, common, embarrassing failure: tests that look like they prove something and prove nothing.

7.3 A leak we found by measuring

The first version of the circuit carried the secret in two columns held *constant* across the whole trace. This looked harmless. It was not. A STARK opens the prover's data at random points, and a constant column has a constant low-degree extension, so the secret showed up, verbatim, in nearly every opening. We knew only because a test checked, over many proofs, whether the raw secret bytes appeared in the proof. They did, twenty times out of twenty. Confining the secret to the eight rows where it is actually needed fixed it, zero out of twenty. A single-sample test would have missed it. This is why we say "the witness is not on the wire" and *not* "zero-knowledge." The prover does not randomize the witness, so formal zero-knowledge is a claim we decline to make.

8 One crowd, one proof

The privacy argument rests on a crowd. If k members do the same action in the same round, an observer's chance of pinning any one execution is at best $1/k$. The natural way to run this is a synchronized round: many members acting identically at one moment, so there is no timing and no ordering to tell them apart.

A small observation with a big payoff: that synchronized round is exactly the shape that lets many proofs collapse into one. A STARK is a statement about an execution table. The k members' tables are independent, so they lay end to end into one longer table, with a *seam* at each boundary that switches off the linking logic so one member's last row does not bleed into the next member's first. One proof for the whole round, one verification.

members k	one batched proof	k separate proofs	saving
4	19,772 B	45,300 B	2.3×
64	43,684 B	1,074,169 B	24.6×

At k=64 a per-member scheme ships over a megabyte of proof and needs 64 verifications. The batched round is one proof, one verification.

Two limits. The prover of a batched round must know every member's secret. For a crowd of distrustful strangers that is a custodian, not privacy, and would need distributed proving. But for one operator running k identities, an agent fleet, a market maker, one prover already holds all k secrets, and the 24.6× is real. And batching trades prover time for proof size. The table is k times longer and proving is super-linear, so a round costs more wall-clock than the sum of separate proofs. It is the right trade when the scarce resource is on-chain verifications.

9 The ricorso: the crowd starts over

The name earns its keep here. *Finnegans Wake* runs on a cycle that returns to its start. RIVERRUN borrows that: the anonymity set is reborn each cycle.

The leak this closes is easy to miss. A member's per-round nullifier changes every round, so one execution is unlinked from another. But their *leaf*, Rescue(s, a), never changes. Published once at commit, it stays forever. Two consequences. Anyone who ever learns your leaf, a bug, a subpoena, a careless client, links you across every round you ever acted in. The nullifiers protect executions from each other, but nothing protects a member from their own history. And a set that only grows is a public timeline of who joined when.

The ricorso lets the set start over each cycle. A member holds a fresh secret per cycle, s_c = Rescue$(s, c, \mathsf{dom}_{\text{cycle}})$, and a fresh leaf ℓ_c = Rescue(s_c, a), and at each rebirth proves the new leaf descends from *some* leaf under the previous root, without revealing which. A migration nullifier, spent once per cycle, stops one seat from becoming several. The two domain tags must differ, or publishing a migration nullifier would leak the next cycle's secret, and a test pins that. History stops piling up.

> The relation is built and mutation-tested *in the clear*. The STARK that proves it without the witness is specified, not built. The reason is concrete: the migration circuit needs five hash cycles before the path, which shifts valid set sizes to $\{8, 2048\}$ while the execution circuit needs $\{4, 64, 16384\}$, and the two do not overlap. Shipping it means realigning both, a day on the most delicate code, and the honest place to stop, with the execution path green, is at the specification. It is exactly where the nullifier binding stopped before it was later built and verified.

10 The ruler: your k is not your k

Everything so far builds the layer and the pool. This section *measures* the anonymity they deliver, and it applies not just to RIVERRUN but to every anonymity set of this kind. It is the part that keeps the strong claims honest, because it names a number that is quietly wrong across a whole category of systems.

10.1 The advertised number counts the wrong thing

Every pool reports its privacy as $1/k$: k members, so a one-in-k guess. That counts *members*, and assumes they are interchangeable. They are not, because of where their money came from. Every member funded their wallet somehow, and the funding trail is public. Trace it backward and you often reach an attributable origin: an exchange's withdrawal address, a known service, a doxxed funder. Sort the crowd by these origins. If the acting identity's origin is visible too, and a fresh withdrawal wallet has to be funded from *somewhere*, then learning it does not leave k suspects. It leaves only the members who share that origin. The crowd you are hidden in is your *provenance class*, not the whole set.

> This is the same shape as the paper's opening point. There is a *surface everyone watches* (the advertised k) and a *larger real surface behind it* (the effective k, set by the funding graph). Guarding the first while ignoring the second is the mistake, and here we can measure exactly how big the gap is.

The measure itself is not new. Serjantov and Danezis [1] showed in 2002 that the honest quantity is not set size but the *entropy* of the distribution linking suspects to the event. Measuring advertised-versus-real anonymity is an established programme on Ethereum [3, 4, 5, 6]. Two things are new here: the channel (funding provenance, which those works do not condition on) and the chain (none of it has run on Solana).

10.2 The formula, and why its direction matters

Split the k members into provenance classes of sizes $n_1, n_2, \ldots$. The adversary learns the actor's class c with probability n_c/k, then guesses uniformly among that class's n_c members. The uncertainty left over is the average over classes of the log of the class size:

$$H = \sum_c \frac{n_c}{k} \log_2 n_c, \qquad k_{\text{eff}} = 2^H. \tag{1}$$

k_{eff} is the size of the crowd the member is genuinely hidden in. One class holding everyone gives $\log_2 k$: nothing was split. All-singleton classes give $H = 0$, an effective k of one: every member identified.

Worked by hand. Ten members: six funded from exchange A, three from B, one alone. Advertised anonymity 1/10. Condition on origin. With probability 6/10 the actor is an "A" and the adversary cannot tell which of six: $\log_2 6 \approx 2.58$ bits. With 3/10, a "B": $\log_2 3 \approx 1.58$. With 1/10, the lone member: 0 bits, identified. Average: $H = 0.6(2.58) + 0.3(1.58) + 0.1(0) = 2.02$ bits, so $k_{\text{eff}} \approx 4.1$. The pool sold ten and delivered four, and the lone member has an anonymity of exactly one, no matter how big the crowd behind them.

The *direction* is the whole thing, and the first implementation had it backwards. It is tempting to measure how *concentrated* the classes are, but that reports catastrophe precisely when the measurement found nothing to split on. A single class of seven, the happy case, came out as "effective $k = 1$", when the honest reading is "effective $k = 7$." Following Serjantov and Danezis, we use the residual entropy of Eq. 1 directly. Four tests pin the direction.

10.3 The measurement, on live mainnet

We ran this against a live Tornado-style SOL privacy pool on Solana mainnet, sampling 30 real depositors and tracing each funding graph backward with a bounded, public-RPC walk.

depositors sampled	30
reach an attributable origin	11/30 (37%), mean 1.18 hops
provenance classes	12
advertised k	**30**
effective k	**6.5**
worst case	**1**

A set advertising 30 delivers an effective 6.5. One depositor sits alone in their class: for them the pool provides no anonymity against an adversary who reads the funding graph, while its dashboard reports one-in-thirty.

Three honesties. It is not a flaw in that pool: no deposit-pool design controls where its users' money came from, which is exactly why the channel goes unmeasured and keeps working. It is a *floor*: a public RPC following only SOL to bounded depth sees less than a funded analyst with a tag database, so the real leak is at least this large. And 37% sits in the same 20–35% range that the Ethereum literature [5] finds with behavioral heuristics on Tornado, a sanity check on the tool, not the pool.

The ruler runs against mainnet today. A single-wallet trace, verbatim from a run over the public mainnet-beta endpoint:

```
{"endpoint":"...mainnet-beta...", "attributable_origin":false, "trace_depth":3, "reliable":true,
"rpc_calls":1, "rpc_failures":0}
```

A deeper 30-depositor audit on the same free endpoint comes back `reliable:false` with a nonzero `rpc_failures`, because the public RPC rate-limits it. The tool marks such a result *partial* rather than reporting a smaller number as fact. A rate-limited or unreachable RPC can never be mistaken for a clean "private" answer. A paid endpoint completes the deep audit.

10.4 The same ruler, turned on RIVERRUN and on the user

Measuring others' pools and not your own is marketing. The same function runs over RIVERRUN's own constructions, against 2000 synthetic members. There are three, forming a ladder of increasing privacy:

construction	reaches an origin	"which origin"	k_{eff} of 2000
rooted decoy (the field)	100%	0 bits	500
cyclic, **ambiguous** origin	100%	**2.90** bits	2000
cyclic, **rootless**	0%	(none)	2000

The first row is what the field ships: every trail ends at one attributable origin, so there is no doubt where the money came from, and the crowd collapses to 500. The third row is the circularity idea: a funding cycle with no beginning, so there is no origin to trace, and the set is worth its full 2000.

The middle row is the strongest one in practice. The funding trail *does* reach origins, 100% of the time, but it reaches *many*, arranged so none is privileged, so "which is the real origin" is not hidden, it is *undefined*. We measure 2.90 bits of doubt about which, and the set stays worth its full 2000. This matters because rootlessness is an idealization. It holds only while a construction's funding sources are themselves unattributable, and the moment one is a known exchange, the "no origin" claim fails. But it does not fail to nothing. It degrades exactly to *ambiguity*. Ambiguity is where rootlessness lands when it meets the real world, and it is where the defense actually lives.

Ambiguity defends a specific thing: attribution to a *particular* origin, not membership in the *group* of origins. If an adversary needs only "this came from one of these three exchanges" rather than which, ambiguity does not stop them. It buys 2.90 bits of "which", not membership secrecy.

The most useful turn is toward the user. The measurement above audits a pool after the fact. Flip it: before a user deposits into *any* pool, trace *their* wallet, sample the pool's depositors, and tell them the anonymity *they* personally would get. The tool is protocol-agnostic. It reads public chain data, not the pool's internals, so it works against a program it has never seen. Three verdicts, each with an action: **Exposed** (alone in your class; fund from a common origin or wait), **Weak** (a small minority shares your class), **OK** (a healthy fraction does, "no cheap attribution found", never "anonymous"). Auditing pools reaches auditors. Protecting a user at the moment they act reaches every user of every privacy protocol on the chain.

11 The coordinator: forming a good crowd on purpose

Here is the payoff of being able to measure. Every other privacy tool treats the crowd as given. RIVERRUN can *measure* a crowd's anonymity, so an agent can *form* a good one on purpose.

The rule is backwards from intuition until you have the formula, then obvious. A crowd funded from the *same* origin is **strong**: an adversary who learns "the actor's money traces to Coinbase" has learned nothing, everyone's does. A crowd of *distinct* origins is **weak**: learning the origin names the one member who has it. So the agent groups members who *share* an origin, and holds back the singletons who would be exposed. Diversity is the enemy here, not the friend.

The policy is a floor. `coordinate(pending, k_min)` admits a member only if at least $k_{\min}$ pending members share their provenance class, and defers the rest with a concrete remedy (wait for more same-origin members, or re-fund from an origin the round already has).

Property 11.1. Every admitted member ends in a class of size $\geq k_{\min}$, so no admitted member is exposed, and among all rounds with that property this one is the largest, since it admits every safe member.

Ten pending members: classes A=5, B=3, and two singletons C, D. Admit everyone (round of 10): $k_{\text{eff}} \approx 3.1$. Coordinate with $k_{\min}$=2 (round of 8, C and D deferred): $k_{\text{eff}} \approx 4.1$. **Fewer members, more anonymity, and nobody exposed.**

Run as an agent loop, intents arrive over time and the agent fires a round the moment a same-origin crowd is ready, holding the rest until they are safe, or forever, if their origin stays rare. Refusing to act, when acting would expose you, is the privacy-preserving choice, and an autonomous agent acting on-chain should make it.

12 Certificates you can check without trusting the agent

The coordinator's claim, "this round is private", should not require trust. A member about to join has to know the agent computed honestly and was not buggy or adversarial. So each fired round carries a *proof-carrying certificate*.

The insight: a proof is not a moat, because a competent engineer can replicate it. What is a moat is making the proof **inescapable** (you cannot claim a bound the evidence does not support), **portable** (a small artifact a stranger checks alone), and **bound to the deployment** (tied to the exact thing it certifies). A certificate turns "trust me" into "check my work."

When the agent fires a round it emits a certificate carrying the admitted set's provenance-class histogram, the floor $k_{\min}$, and the round's k_{eff}. Anyone verifies it. **Inescapable:** verification recomputes k_{eff} from the histogram and checks $\min_c n_c \geq k_{\min}$, so a coordinator cannot certify a bound the histogram does not support, and a tampered number fails (tested). **Portable:** the certificate is a small object a member, an auditor, or a settling program checks without re-tracing the graph. **Bound to the round:** a blake3 commitment ties it to the exact admitted set, so it cannot be replayed onto a different crowd (tested). The certificate carries only the histogram, never member identities: it proves the crowd meets the floor without revealing who is in which class.

13 On-chain: what runs, measured

A privacy tool nobody can afford to run is a paper, not a tool. So we measured.

set size	proof	prove	verify	setup artifacts
4	11,929 B	2.0 ms	0.20 ms	0 bytes
64	17,045 B	3.7 ms	0.35 ms	0 bytes
16,384	19,064 B	8.1 ms	0.43 ms	0 bytes

A set of 16,384 members proves in 8 ms. Proof size grows logarithmically: 4096× the members costs 1.6× the proof. The last column is the point: nothing to distribute, nothing to trust.

The real question is whether the *chain* can check the proof. If it can, no one is trusted. If not, someone is. There are two proof systems in play, and they land in two different places.

The Winterfell STARK (the research core). We built a Solana program that runs the real Winterfell verifier, FRI, Merkle openings, Fiat–Shamir, constraint evaluation, inside the runtime, and measured it. It runs, and it costs 3.77M compute units at k=4. That exceeds Solana's 1.4M per-transaction cap, so it needs execution chunked across transactions. Sweeping the FRI queries to the floor (4→1.57M CU) does not get under the cap even at a security level no one would ship. So single-transaction verification of *this* proof is not a tuning problem. It is a *field* problem: the 128-bit field is too big.

The M31 Circle STARK (the fix, now proven in a VM). The answer, argued in the last version of this paper as a roadmap, is now demonstrated. A Circle STARK over the 31-bit field M31, the field production provers use [16], with an existing on-chain Solana verifier [17], post-quantum, no

ceremony, shrinks both proof and cost dramatically.

Using the M31 Circle-STARK verifier of [17], we proved and verified RIVERRUN's relation, membership plus nullifier plus a bound public action, inside a Solana VM (LiteSVM). A real 8,696-byte proof is **accepted at** 159,849 **compute units**, about 11% of the 1.4M cap, in a single transaction. A proof carrying a *different* action is **rejected**. This is the whole verification, on-chain-class, under the cap.

Two honest limits. This ran in a local Solana VM (LiteSVM), not yet on mainnet, and the relation it verifies is riverrun's membership-plus-nullifier-plus-action, not yet the full in-circuit Poseidon2 Merkle path (the hash foundation is built on Plonky3's vetted parameters, and the arithmetization is the next step). Until that lands on mainnet, the deployed program does not verify the proof itself. Rather than leave that implicit, it makes the trust explicit, bounded, and *distributed*: the pool names a committee of N verifier keys and a threshold M, and an execution must carry at least M signatures from *distinct* committee members over exactly the tuple being settled, checked through the runtime's signature precompile. This removes the two cheap attacks, an arbitrary signer settling an action and a watcher front-running a nullifier, and prices the rest: a false attestation now needs M colluding verifiers, not one. Five on-chain tests cover it (a 2-of-3 quorum settles, one vote alone does not, a member signing twice counts once, an outsider does not count). The M31 path above is how the trust goes to zero.

13.1 A self-audit, with every fix carried by a test

Before claiming any of this is safe, we attacked it. A security pass over the settlement and identity code closed three real findings, each with a regression test.

- **Recipient redirection on the verified path (high).** The STARK-verified settlement bound the root, nullifier, round, and action, but *not* the payout recipient, so the settling relayer, the untrusted party, could name its own account and redirect the funds. The recipient is now a verified public input, checked against the account being paid. The committee path already bound it.
- **RLN action point (medium).** The rate-limit share point must be unpredictable and unique per action, or the limit is evadable, and the value 0 would publish the secret directly. It is now derived from the action itself and never zero.
- **Recovery panicked on a crafted transcript (low).** An adversary could feed duplicate points and crash the recovery. It now returns an error.

A read-audit of the STARK confirmed the membership, nullifier, and action weld and the batched-round seam hold, with the negatives tested: another member's nullifier, a different action, a cross-member claim. This does not replace an independent formal audit, still on the roadmap. But the findings it surfaced are closed, not noted.

14 The mathematics of anonymity

The systems above use the effective anonymity-set size and the repeated-use bound as tools. This section derives them from first principles, states them as theorems, and gives the arguments. Nothing here is folklore we ask you to accept. It is the formal backbone under §10 and §5.2.

14.1 Anonymity is uncertainty, measured in bits

Fix an adversary and a single observed act. Let $\Psi = \{u_1, \ldots, u_N\}$ be the population of possible actors, and let the adversary's knowledge, after seeing everything it can see, be a probability distribution $P = (p_1, \ldots, p_N)$ over who performed the act, with $\sum_i p_i = 1$. Perfect anonymity is $p_i = 1/N$ for all i: the adversary learned nothing, and every actor is equally likely. Perfect exposure is $p_j = 1$ for some j: the adversary is certain. Anonymity is the distance between these, and the right way to measure the uncertainty in a distribution is its **Shannon entropy** [19],

$$H(P) = -\sum_{i=1}^{N} p_i \log_2 p_i,$$

the expected number of bits an adversary still lacks to name the actor. Serjantov and Danezis [1], and independently Diaz et al. [20], proposed this in 2002 as the honest metric, against the naive count N. The **effective anonymity-set size** is the entropy exponentiated,

$$k_{\text{eff}} = 2^{H(P)},$$

the size of the uniform crowd that would give the same uncertainty. If P is uniform on a subset of size m and zero elsewhere, then $H = \log_2 m$ and $k_{\text{eff}} = m$, so on the easy case the metric agrees with counting. On every other case it corrects it.

> Why entropy and not the count. Two pools each advertise $k = 4$. In the first, the adversary's posterior is uniform, $(\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4})$, and $k_{\text{eff}} = 4$. In the second, it is $(\frac{7}{10}, \frac{1}{10}, \frac{1}{10}, \frac{1}{10})$, one member far more likely than the rest. The count says both are 4. The entropy says the second is $H \approx 1.36$ bits, $k_{\text{eff}} \approx 2.6$: the second pool delivers barely two-thirds of the anonymity of the first, and the count could never see it. The gap between what a pool advertises and what it delivers lives entirely in this difference.

14.2 Conditioning on provenance: the effective-k formula

The count assumes members are interchangeable. They are not, because the funding graph is public (§10). We derive the correction. Partition the k members into **provenance classes** $C_1, \ldots, C_t$ by attributable funding origin, with sizes $n_1, \ldots, n_t$ and $\sum_c n_c = k$. Model the adversary as follows. The true actor is uniform over the k members a priori. The adversary observes the acting identity's provenance class, which is cheap because a fresh withdrawal wallet must be funded from somewhere visible. Conditioned on that observation, the actor is uniform over the members of the observed class and impossible elsewhere.

Then the actor lies in class C_c with probability n_c/k, and given that, the adversary's residual entropy is $\log_2 n_c$, the uncertainty within a uniform class of size n_c. The expected residual entropy is

$$H \;=\; \sum_{c=1}^{t} \frac{n_c}{k} \log_2 n_c, \qquad k_{\text{eff}} \;=\; 2^H. \tag{2}$$

This is Eq. 1, now derived rather than asserted. It is a conditional entropy, $H(\text{actor} \mid \text{observed class})$, and it is exactly the average over classes of the log of the class size.

Claim 14.1 (Bounds and boundary cases)**.** For any partition of k members into classes, $0 \le H \le \log_2 k$, so $1 \le k_{\text{eff}} \le k$. The upper bound $k_{\text{eff}} = k$ holds iff a single class contains everyone (nothing was learned). The lower bound $k_{\text{eff}} = 1$ holds iff every class is a singleton (every member is identified). If any one class is a singleton, that member has $k_{\text{eff}} = 1$ regardless of how large the other classes are.

Argument. H is a nonnegative sum of nonnegative terms, and each term $\frac{n_c}{k} \log_2 n_c$ is zero exactly when $n_c = 1$. So $H = 0$ iff every $n_c = 1$, giving $k_{\text{eff}} = 1$. For the upper bound, H is the conditional entropy $H(A \mid C)$ of the actor A given the class C, and conditioning cannot increase entropy above the unconditional $H(A) = \log_2 k$ of a uniform actor, with equality iff C is independent of A, which for this partition means a single class. The singleton remark is immediate: a member alone in their class is named the moment their class is observed, and the per-member anonymity is a minimum, not the average k_{eff}, which is why the ruler reports the worst case alongside the mean. □

The *direction* matters and the first implementation had it backwards (§10). One is tempted to score concentration by **min-entropy**, $H_\infty = -\log_2 \max_i p_i$, which measures the single most likely suspect. But min-entropy reports catastrophe precisely when the partition found nothing to split on: a single class of seven, the safest case, has one most-likely-class probability of 1 and $H_\infty = 0$, reading as $k_{\text{eff}} = 1$ when the honest answer is 7. Shannon entropy, the *average* residual, does not have this failure, and is the quantity Eq. 2 uses. Min-entropy answers a different and also-useful question, "how exposed is the worst-off member," which the ruler reports as the worst case, but it is not the set's anonymity.

14.3 The repeated-use bound: an intersection attack, formally

Single-shot k_{eff} scores one act. A riverrun-ID identity is one secret used many times (§5.2), and Danezis warned that entropy metrics are "very poor" at repeated use [2]. We formalize why, and give the correct bound.

Setup. A persistent user u acts in contexts $\theta_1, \ldots, \theta_n$ through pseudonyms that are cryptographically unlinkable. The adversary cannot link the pseudonyms. But u carries a *persistent quasi-identifier*, most concretely their funding origin, which recurs under every use and is public. In context i the adversary observes a feature $X_i = \varphi_i(u)$ and forms the candidate set consistent with it, $S_i = \{v \in \Psi : \varphi_i(v) = X_i\}$, which always contains the true u. Linking the acts by the recurring quasi-identifier rather than the pseudonym, the adversary intersects:

$$T_n \;=\; \bigcap_{i=1}^{n} S_i, \qquad k_{\text{eff}}^{(n)} \;=\; |T_n| \text{ (uniform posterior on } T_n).$$

Property 14.1 (Monotone erosion). $k_{\text{eff}}^{(n)}$ is non-increasing in n: $k_{\text{eff}}^{(n)} \le k_{\text{eff}}^{(n-1)} \le \cdots \le k_{\text{eff}}^{(1)}$. More uses never increase anonymity, and $k_{\text{eff}}^{(1)}$ is exactly the single-shot number of §10. The single-shot metric is therefore an *over*-estimate of a persistent identity's true anonymity.

Argument. $T_n = T_{n-1} \cap S_n \subseteq T_{n-1}$, so $|T_n| \le |T_{n-1}|$. The base case $T_1 = S_1$ is the single-shot candidate set. Intersection is monotone decreasing under inclusion, which is the whole content. □

The differential-privacy view. Read each use as a query leaking information about u. Let $\varepsilon_i = H(u \mid X_{<i}) - H(u \mid X_{\le i}) \ge 0$ be the bits that use i removes, its privacy loss. The losses telescope:

$$\sum_{i=1}^{n} \varepsilon_i \;=\; H(u \mid \varnothing) - H(u \mid X_{1..n}) \;=\; \log_2 |\Psi| - \log_2 k_{\text{eff}}^{(n)},$$

so

$$k_{\text{eff}}^{(n)} \;=\; |\Psi| \cdot 2^{-\sum_i \varepsilon_i}. \tag{3}$$

A persistent identity's anonymity **decays exponentially in its accumulated per-use leak**. This is the composition theorem of differential privacy [21] applied to anonymity: leaks add, and the crowd shrinks by the exponential of the sum. Fix an anonymity floor $k_{\min}$. The identity is safe under one secret while $\sum_i \varepsilon_i \le \log_2(|\Psi|/k_{\min})$, and the moment the sum crosses that budget, $k_{\text{eff}}^{(n)} < k_{\min}$.

Claim 14.2 (The rotation trigger). When the budget is spent, the correct action is already a power of the primitive: the **turn** (§5) rotates the secret to a fresh lineage whose candidate set S is un-intersected, resetting the accumulated crypto-linkage to zero. The metric does not merely warn, it fires the defense.

Two honest bounds on this result. First, **turn** resets the cryptographic lineage, not the physical quasi-identifier: if the next lineage is funded from the same rare origin, the origin-based intersection survives the rotation, so the two defenses must compose, rotate the secret *and* re-fund from a common origin. Second, Eq. 3 uses basic composition, a worst-case upper bound on leak; correlated features can erode faster in the tail, and advanced composition gives a tighter typical-case $\sqrt{2n \ln(1/\delta)}\,\varepsilon$ under an independence assumption we do not claim holds. The number is a floor under a stated observation model, in keeping with the ruler's discipline everywhere else.

14.4 The anonymity trilemma, and where a synchronized round pays

There is a lower bound on what anonymity costs, and it explains the shape of riverrun's pool. Das, Meiser, Mohammadi, and Kate proved an **anonymity trilemma** [44]: a protocol cannot simultaneously provide strong anonymity, low latency, and low bandwidth overhead. Strong anonymity against a global observer requires either that messages wait (latency, so that many are in flight together and cannot be ordered) or that cover traffic is sent (bandwidth, so that real messages hide among decoys). You may have any two of the three; the third pays.

riverrun's synchronized round (§8) is a deliberate choice of which corner to pay. It spends **latency**: members wait to act together at one moment, so there is no timing or ordering for an observer to

separate them, which is exactly the "many in flight together" the trilemma says buys anonymity. It does not spend bandwidth on cover traffic, because the crowd is other real members rather than manufactured decoys, which is why forming a genuine same-provenance crowd (the coordinator, §11) matters more here than in a decoy-based design: with no chaff, the anonymity is exactly the real crowd, no larger and no smaller, which is precisely what the ruler measures. The trilemma is the reason a privacy pool cannot be instant, free, and strong at once, and riverrun's answer is to be strong and lean by asking members to synchronize, an honest cost stated rather than hidden.

15 The mathematics of post-quantum security

§4.1 argued that a permanent-ledger privacy tool must be post-quantum. This section supplies the cryptography under that argument: what a quantum computer breaks, what it does not, and why riverrun's hash-only design is on the safe side by a margin that is quantified, not hoped.

15.1 What Shor breaks, and why the curves fall

The security of elliptic-curve cryptography (ECDSA signatures, ECDH key exchange, pairing-based SNARKs) rests on the hardness of the discrete-logarithm problem: given g and g^x in a group, recover x. Classically the best attacks are exponential in the key size, which is why 256-bit curves are considered safe today. Shor's algorithm [32] solves discrete logarithm, and integer factorization, in *polynomial* time on a quantum computer, by reducing each to period-finding and solving that with the quantum Fourier transform. The consequence is total, not marginal: a cryptographically-relevant quantum computer does not weaken curve-based schemes, it breaks them, recovering private keys from public ones. Every construction whose privacy or soundness rests on discrete log, including the pseudonym unlinkability of deployed BBS credentials (§5.1), falls at once.

15.2 What Grover does to a hash, and why it is survivable

Hash functions rest on different assumptions: preimage resistance (given y, find x with $H(x) = y$) and collision resistance (find $x \neq x'$ with $H(x) = H(x')$). Against these the relevant quantum attack is Grover's [33], an unstructured search that finds a marked item among 2^b in $O(2^{b/2})$ steps, a *quadratic* speedup, not an exponential one. The effect on security is precise and bounded: Grover halves the effective preimage security of a b-bit hash from b to $b/2$ bits. A 256-bit hash retains 128 bits against a quantum adversary, which is the standard target. Collision resistance is affected even less in any practical model, and the 128-bit security parameters riverrun's proofs target (§13) already absorb the Grover loss.

Claim 15.1 (Why hash-only is post-quantum)**.** Every value riverrun commits to a permanent ledger, the commitment $\text{Rescue}(s, a)$, the nullifier $\text{Rescue}(s, r)$, the Merkle root, the pseudonyms of riverrun ID, is the output of a collision-resistant hash. Their unlinkability and binding reduce to preimage and collision resistance of that hash, for which the only known quantum attack is Grover's quadratic speedup, absorbed by the parameters. No value on the permanent record reduces to discrete logarithm or factoring. Therefore no value on the permanent record is broken by Shor's algorithm, and the anonymity is *everlasting* in the sense of §5.1: unrecoverable from today's transcript by any future quantum adversary.

15.3 The Mosca inequality, and the harvest-now corollary

The timing argument of §4.1 deserves a formal statement. Following Mosca [34], let X be the required security lifetime of a secret, Y the time to migrate a system to quantum-safe cryptography, and Z the time until a cryptographically-relevant quantum computer exists.

Claim 15.2 (Mosca's inequality)**.** If $X + Y > Z$, the system cannot protect its secrets for their required lifetime: the quantum computer arrives before migration completes, and any data recorded before migration and still sensitive is exposed retroactively.

Claim 15.3 (The permanent-ledger corollary)**.** For data written to a public, permanent ledger, the anonymity requirement has no expiry, so $X = \infty$. Then $X + Y > Z$ holds for every finite Z, regardless of migration speed. Therefore a privacy tool whose protected data lives on a permanent public ledger cannot satisfy Mosca's inequality unless its primitive is *already* quantum-safe at the moment of writing. Post-quantum security is not a risk to manage for such a tool; it is a necessary condition.

Harvest-now-decrypt-later is the corollary made operational. The adversary needs no quantum computer today. Copying the entire public ledger is free and requires nothing but storage. The computation that breaks it is deferred to the day the hardware exists, at which point every curve-based guarantee ever written to the chain fails at once, including guarantees written in 2026 by tools whose authors reasoned that the hardware was still years away. The direction of the argument is the point: for permanent data, the arrival date of the quantum computer is irrelevant to whether you are safe, and relevant only to *when* you are broken. The standards bodies have drawn the same conclusion for data far less permanent than a ledger: NIST finalized FIPS 203, 204, and 205 in August 2024, and its hash-based signature SLH-DSA (FIPS 205) rests on the same conservative assumption riverrun does.

16 The algebra of accountable anonymity

Anonymity without accountability invites abuse: a single actor floods a system while hiding in the crowd. riverrun's answer, the nullifier and the rate-limiting construction, is algebra, and this section states it. The theme is Chaum's from §3: prevent misbehavior without naming the honest.

16.1 The nullifier as a pseudo-random function

A nullifier is $n = \text{Rescue}(s, r)$, a hash of the secret s and the round label r. Treated as a pseudo-random function keyed by s, it has the two properties accountability needs. **Uniqueness**: for fixed s and r, n is determined, so a member cannot produce two distinct valid nullifiers for one round, which is what enforces one act per member per round. **Unlinkability**: n reveals nothing about s under preimage resistance, and $n = \text{Rescue}(s, r)$ and $n' = \text{Rescue}(s, r')$ for $r \neq r'$ are unlinkable under the PRF assumption, so a member's acts across rounds cannot be tied together. This is the deterministic-nullifier primitive that PLUME [7] formalizes and proves, and it is why the pool can enforce a rate without a registry of identities. The subtle requirement, that the revealed n be bound to the *same* s that proved membership, is the weld of §7, enforced inside one proof rather than by comparing two.

16.2 Rate-limiting by Shamir secret sharing

To allow N anonymous acts per context but unmask the N+1-th, riverrun uses Shamir secret sharing [35] in the RLN construction [37]. Work over a prime field $\mathbb{F}_p$; riverrun uses the Mersenne prime $p = 2^{61} - 1$, small enough that products fit a 128-bit word and large enough that a hash-derived point is effectively uniform. For a context θ with limit N, the holder's identity s is the constant term of a degree-N polynomial

$$P(x) \;=\; s + a_1 x + a_2 x^2 + \cdots + a_N x^N,$$

whose other coefficients $a_i = H(s \,\|\, \theta \,\|\, i)$ are derived deterministically from the secret and the context, so the holder commits to one polynomial per context and cannot equivocate. Each act publishes one point $(x, P(x))$, where the abscissa x is derived from the act itself, $x = H(\theta \,\|\, \text{action})$, and not chosen freely.

Claim 16.1 (The N+1 threshold)**.** A degree-N polynomial is determined by any N+1 distinct points and under-determined by N or fewer. Therefore up to N acts leave $s = P(0)$ information-theoretically hidden: infinitely many degree-N polynomials pass through N points, with every field element equally possible as the constant term. The $(N+1)$-th act supplies the pinning point, and anyone can recover s by Lagrange interpolation at zero,

$$s \;=\; P(0) \;=\; \sum_j y_j \prod_{m \neq j} \frac{-x_m}{x_j - x_m},$$

unmasking the over-actor. Spam is priced in the spammer's own de-anonymization.

Two properties of x are load-bearing, and one was an audit finding (§13.1). The abscissa x must be **unpredictable and unique per act**: if the actor could reuse an x, they could act repeatedly while publishing only N distinct points and never crossing the threshold, defeating the limit; deriving x from the action's content forces distinct acts to distinct points. And x must be **nonzero**, since $P(0) = s$, so a share at $x = 0$ would publish the secret directly; the derivation is remapped away from zero. Cross-context safety follows from the per-context coefficients: points from different θ lie on different polynomials and do not combine, so acting once in each of many contexts is not over-acting.

17 The seven powers, formally

§5 described riverrun ID's seven powers in words. Each is a precise relation over the secret and a public context, and each is a tested function in the core crate. We state them as statements and witnesses, in the style of §23, so that "the user can do X" means "there is a relation the holder can satisfy and no one else can." Throughout, s is the secret, θ a public context, and H a domain-separated hash (each power uses its own tag, so a value in one role can never be reinterpreted in another).

1. **shape.** $\mathsf{shape}(\theta) = H(\text{"shape"} \,\|\, s \,\|\, \theta)$. The published per-context identity, a Merkle leaf for that context's set. Unlinkability: $\mathsf{shape}(\theta_1)$ and $\mathsf{shape}(\theta_2)$ are independent hash outputs of one secret, so linking them reduces to breaking the hash (§24, everlasting).

2. **fit.** $\mathsf{fit}(\theta) = H(\text{“fit”} \,\|\, s \,\|\, \theta)$. The per-context nullifier, revealed once to act, spent-once by the on-chain registry. It is the accountability half of §16: unique per context, unlinkable across, and tied to the same s that proves membership by the weld.
3. **turn.** $\mathsf{turn}(\theta) = H(\text{“turn”} \,\|\, s \,\|\, \theta \,\|\, \theta{+}1)$. Continuity. The relation CheckTurn has statement $\{R_{\text{prev}}, \theta, \tau\}$ and witness $\{s, \text{path}\}$, and holds iff $\tau = \mathsf{turn}(\theta)$ and $\mathsf{shape}(\theta)$ is a leaf under R_{prev}. It proves “the same holder who was a member last cycle is rotating forward,” revealing τ but not which member, and is the primitive the ricorso and the erosion-rotation of §5.2 spend. It is proven in zero knowledge over the STARK, with its negatives (a forged tag, a tag from another angle) rejected in tests.
4. **link.** Relation CheckLink, statement $\{\mathsf{shape}_a, \theta_a, \mathsf{shape}_b, \theta_b\}$, witness $\{s\}$, holds iff both shapes derive from the same s. It lets the holder, and only the holder, prove that two of their identities are one, to a chosen verifier, for chosen contexts. This is contextual integrity’s “join two contexts when the norm permits” (§2) made checkable.
5. **grant.** $\mathsf{grant}(\theta, d) = H(\text{“grant”} \,\|\, s \,\|\, \theta \,\|\, d)$ for a delegate key d. Relation CheckDelegation, statement $\{R, \theta, d, g\}$, witness $\{s, \text{path}\}$, holds iff $g = \mathsf{grant}(\theta, d)$ and the holder is a member. It authorizes d to act as the holder in context θ and only there, bound to d (another key cannot use it) and scoped to θ (it grants nothing elsewhere), without exposing s. This is the delegatable-credential power, the narrowest safe way to lend an identity to an agent.
6. **rln.** The Shamir share $(x, P(x))$ of §16, with x derived from the action. Statement: the published points; witness: the secret polynomial. Below N points it is perfectly hiding; at $N{+}1$ it recovers s. Accountable rate-limiting with no trusted party.
7. **credential.** $\mathsf{cred}(\mathsf{attr}) = H(\text{“cred”} \,\|\, s \,\|\, \mathsf{attr})$ for an issuer-signed attribute. Relation CheckAttribute, statement $\{R_{\text{attr}}, \mathsf{attr}, \theta, \mathsf{shape}\}$, witness $\{s, \text{path}\}$, holds iff the holder’s credential for attr is under the issuer’s attribute root and binds to their context identity. It shows “verified,” “over 18,” carried per context without a trail joining the showings.

Property 17.1 (One secret, one control surface)**.** All seven relations read the same s and differ only in their domain tag and public inputs. Consequently a holder can satisfy any subset of them simultaneously for one identity, and no party lacking s can satisfy any, while the domain separation guarantees that a value produced for one power (a fit, say) is never a valid value for another (a grant). The unlinkability of **shape** composes with the accountability of **fit** and **rln** because they are independent hashes of the same secret: the adversary sees a fit and a shape and cannot tie them without s, yet the holder proves in zero knowledge that they share s whenever a power (turn, link, grant, credential) requires it. This is the formal content of “one secret, full control of your own exposure.”

These are relations, tested in the clear over BLAKE3 in the specification crate (48 tests, the turn additionally proven in zero knowledge over the STARK). What is not yet built is the in-circuit proof of the membership-bearing powers (turn, grant, credential) over the small field, and its Solana verifier (§13). The relations are correct and tested; their succinct on-chain proofs are the road ahead, and we do not present a specification as a deployment.

18 STARK soundness, in full

§7 described the proof informally. This section states what a STARK proves, why a verifier should believe it, and, with equal care, what riverrun's STARK does *not* provide. The honesty about the last point is why we never write "zero-knowledge" without qualification.

18.1 Arithmetization: a computation as a low-degree object

A STARK proves that a computation was performed correctly by encoding it as an **algebraic intermediate representation** (AIR). The execution is a table of w columns (registers) and T rows (time-steps). Correctness is expressed as constraints of two kinds: **boundary** constraints fix specific cells (the first row holds the inputs, a designated row holds the public output), and **transition** constraints require that every pair of adjacent rows satisfies a fixed polynomial relation, so "the machine stepped correctly" becomes "a set of polynomials vanishes on every transition." Interpolating each column over a domain of size T turns the columns into polynomials of degree $< T$, and the constraints become polynomial identities that must hold on the trace domain. A correct execution yields constraint polynomials that vanish on the whole domain, hence are divisible by the domain's vanishing polynomial; a cheating execution does not. The prover's task reduces to convincing the verifier that a certain quotient is a genuine low-degree polynomial rather than an arbitrary function faked to look right at a few points.

18.2 Low-degree testing and FRI

The heart of a STARK is therefore a **low-degree test**: the verifier must check, without reading the whole polynomial, that a committed function is close to a polynomial of bounded degree. This is what makes soundness possible, because a *wrong* low-degree polynomial that agrees with a right one on the trace domain would have to agree almost everywhere, and two distinct polynomials of degree $< d$ agree on at most d points, a vanishing fraction of a large evaluation domain. So a cheat is caught by a few random spot-checks with overwhelming probability. FRI, the Fast Reed-Solomon Interactive Oracle Proof of Proximity [36], performs the test efficiently: it commits to the function's evaluations on a large domain via a Merkle tree, then repeatedly "folds" the function, halving its degree and domain each round by taking a random linear combination of its even and odd parts, until the degree is constant. Consistency between consecutive folds is checked at random points, and each opened point is authenticated by its Merkle path. A function far from low-degree survives a single folded query with probability bounded below one, so repeating the query drives the soundness error down geometrically.

18.3 From interactive to non-interactive, and the soundness error

FRI is an interactive protocol: the verifier supplies random folding challenges and query positions. The **Fiat-Shamir transform** makes it non-interactive by deriving those challenges from a hash of the transcript so far, so the prover cannot choose its commitments after seeing the challenges. Soundness then holds in the random-oracle model, the standard heuristic for hash-based proofs, and this is the sense in which a STARK's security "rests on a hash." The concrete soundness error is governed by three knobs: the code rate (blowup) ρ, the number of query repetitions q, and proof-of-work grinding g bits added to the transcript. Informally the error scales like $\rho^{\,q} \cdot 2^{-g}$ up to

small-degree factors, so a target of 128 bits is reached by a combination such as a blowup of 8, on the order of tens of queries, and a modest grinding factor. This is why the same relation costs a different number of on-chain compute units at 84-bit and 128-bit parameters (§13): the query count, and therefore the proof size and the verifier's work, is set by the soundness target.

18.4 What a STARK is, and is not

A STARK is **succinct** (the verifier does far less work than re-running the computation), **transparent** (no trusted setup, only a public hash), and, because its only cryptographic assumption is a collision-resistant hash, **post-quantum** in the sense of §15. These three are exactly why riverrun chose STARKs over pairing-based SNARKs, and the price, paid openly, is proof size in kilobytes rather than bytes.

What a STARK is *not*, by default, is **zero-knowledge**. Soundness (a false statement is rejected) and zero-knowledge (a true proof leaks nothing about the witness) are separate properties, and the base FRI-STARK provides the first without the second. Achieving formal zero-knowledge requires randomizing the trace, masking the witness polynomials with random ones so that the openings the verifier sees are independent of the secret. The Winterfell prover riverrun builds on does not do this. We therefore make the weaker, honest claim: the witness is not transmitted on the wire, which we verified empirically by testing over many proofs whether the raw secret bytes appear in the proof (§7.3), and which caught a real leak, a constant witness column whose low-degree extension exposed the secret in nearly every opening, that a formal-ZK construction would have prevented by design. Closing the gap to formal zero-knowledge is named work, not a claim we quietly make.

> Soundness of the *scheme* is not soundness of *our AIR*. The arguments above establish that a correct STARK verifier rejects false statements about a given set of constraints. Whether riverrun's hand-written constraints actually encode the intended relation, membership, nullifier binding, and action binding welded through one secret, is a separate question, and it is the one mutation testing addresses (§7): for each "X is bound" we delete the constraint and confirm a test fails. That discipline found three constraints that were decorative and is how they were fixed. It is necessary, not sufficient, and an independent audit of the arithmetization remains on the roadmap.

19 Membership without identity: Merkle accumulators

The pool and riverrun ID both rest on one question: how does a member prove they belong to a set without revealing *which* member they are? The answer is a Merkle accumulator, and its security is a clean reduction to the hash.

A **Merkle tree** over leaves $\ell_1, \ldots, \ell_m$ builds a binary tree whose each internal node is the hash of its two children, up to a single **root** R. The root is a short commitment to the entire set: it is 32 bytes regardless of m, and changing any leaf changes the root. A member proves that their leaf ℓ is in the set by exhibiting the **authentication path**, the sequence of sibling hashes from the leaf to the root, which lets a verifier recompute the root from ℓ and check it against the public R. The path has length $\log_2 m$, so membership in a set of 16,384 is proved by 14 sibling hashes.

Claim 19.1 (Soundness of membership)**.** Under collision resistance of the hash, no probabilistic polynomial-time prover can exhibit a valid authentication path from a leaf $\ell \notin \{\ell_1, \ldots, \ell_m\}$ to the published root R, except with negligible probability.

Argument. Suppose a prover produces a path from a non-member ℓ that recomputes to R. Walk the claimed path and the honest tree from the root downward. Both begin at the same value R. At the first level where the claimed path's node value differs from the honest tree's node in the corresponding position, the two nodes are distinct inputs, yet by construction their parent hashes are equal, since both paths recompute to the same value above. That is a hash collision. So a successful forgery yields a collision, and by collision resistance this happens only with negligible probability. Hence a valid path certifies genuine membership. □

Two properties make this the right primitive for anonymity. It is **succinct**: the root and the path are logarithmic, so a set of millions is committed in 32 bytes and proved in a few hundred. And it is **identity-hiding when proved in zero knowledge**: the authentication path names the leaf's position and its siblings, which would leak which member acted, so riverrun does not reveal the path but proves *knowledge* of a valid path inside the STARK (§18). The public inputs are the root and the nullifier; the leaf, its index, and the path are the private witness. This is the exact line between a transparent accumulator, which anyone can update and check, and the anonymity that comes from proving membership without exhibiting it.

20 Arithmetization-oriented hashing

A STARK proves statements about arithmetic over a finite field, so every hash inside the circuit must be expressed as low-degree polynomial constraints. Standard hashes such as SHA-256 are built from bit operations, XORs and rotations, which are expensive to write algebraically: representing them as field arithmetic inflates the trace enormously. **Arithmetization-oriented** hashes are designed to be cheap in exactly this setting, and riverrun uses Rescue-Prime [14] in the Winterfell path and Poseidon2 in the small-field path.

Their structure is a substitution-permutation network over the field. Each round applies a nonlinear **S-box**, the map $x \mapsto x^{\alpha}$ for a small α coprime to $p - 1$ (so it is a permutation), a linear mixing layer (multiplication by a fixed MDS matrix that diffuses each element across the state), and the addition of round constants. Rescue alternates the S-box with its inverse $x \mapsto x^{1/\alpha}$, which is high-degree in one direction and low-degree in the other, so that both the forward computation and its verification are efficient. The security argument rests on the algebraic degree growing fast enough over the rounds to resist interpolation and Groebner attacks, and the designers give a round count with a margin. The single knob that matters for cost is that a low α, typically 3 or 5, keeps the per-round constraint degree small, which is why the AIR of §7 declares transition constraints of degree 5: that is the Rescue S-box degree, and it is what bounds the blowup the STARK needs.

The security of an arithmetization-oriented hash depends on its round count and constants, and riverrun uses vetted parameters rather than invented ones: the Poseidon2 instance over Mersenne-31 uses Plonky3's canonical parameters (round numbers and Grain-generated constants), not hand-picked values. Reduced-round parameters buy speed at the cost of security margin, and the Winterfell path's example-grade ≈ 84-bit setting (§28) is a reduced-

round choice we flag rather than hide. A production deployment raises the round count to the 128-bit margin, at the compute cost §13 measures.

21 Small fields and Circle STARKs

The single measured obstacle to on-chain verification (§13) was the field size: over a 128-bit field, one relation costs 3.77M compute units, above Solana's 1.4M cap, and no amount of query tuning brings it under, because the base cost of field arithmetic dominates. The fix is a smaller field, and the mathematics of why deserves a statement.

Arithmetic in a field of b-bit elements costs, per operation, on the order of b or more machine cycles, and a STARK performs a number of field operations that scales with the trace and the query count. Shrinking the field from 128 bits to 31 multiplies neither the trace length nor the query count, but it shrinks the cost of every one of the millions of underlying multiplications, and it shrinks the size of every committed field element in the proof. The field of choice is the **Mersenne prime** $M31 = 2^{31} - 1$, whose modular reduction is a shift and an add rather than a division, making its arithmetic among the fastest available on ordinary hardware and on the Solana VM.

The catch a 31-bit prime field poses, and the reason Circle STARKs exist, is that FRI needs a large multiplicative subgroup of smooth (highly two-adic) order to fold over, and $M31 - 1 = 2 \cdot 3^2 \cdot 7 \cdot 11 \cdot 31 \cdot 151 \cdot 331$ has only a small power of two dividing it, so $\mathbb{F}_{M31}^{\times}$ has no large power-of-two subgroup. **Circle STARKs** [16] resolve this by working not in the multiplicative group of the field but on the **circle group**, the set of points (x, y) with $x^2 + y^2 = 1$ over $\mathbb{F}_{M31}$, which does have a subgroup of order a large power of two. The FRI folding is carried out on this circle, and the low-degree test transfers. The payoff is measured, not argued: riverrun's relation, proved over M31 and verified inside the Solana VM, is accepted at 159,849 compute units, about 11% of the cap, in a single transaction (§13). The move from a 128-bit field to a 31-bit circle is the whole difference between "needs chunking across transactions" and "fits in one."

22 The coordinator is optimal

The coordinator (§11) admits members to a round only when their provenance class is large enough. We stated its guarantee informally; here it is a theorem with a proof, because "forms a good crowd" should mean something exact.

Setup. A set M of pending members is partitioned into provenance classes. A round is a subset $A \subseteq M$. A member is **safe** in A if their class, restricted to A, has size at least $k_{\min}$. The policy `coordinate` admits a member iff at least $k_{\min}$ pending members share their class, that is, it admits exactly the union of all classes whose full size is $\geq k_{\min}$.

Property 22.1 (Soundness and maximality)**.** Let $A^\star$ be the set the policy admits. Then (i) every member of $A^\star$ is safe in $A^\star$, so no admitted member is exposed; and (ii) $A^\star$ is the largest set with property (i): for any A in which every member is safe, $A \subseteq A^\star$.

Argument. (i) $A^\star$ is a union of whole classes each of full size $\geq k_{\min}$, and admitting a whole class does not shrink any member's class within the round, so each admitted member sees at least $k_{\min}$

classmates. (ii) Suppose A has every member safe, and take any $x \in A$ in class C. Safety means $|A \cap C| \geq k_{\min}$, so C has at least $k_{\min}$ members among the pending, hence $|C| \geq k_{\min}$, hence $C \subseteq A^\star$ by definition of the policy, hence $x \in A^\star$. So $A \subseteq A^\star$. □

The theorem says the greedy policy is not a heuristic that usually helps but the *unique* largest safe round: it exposes no one and defers no one it could safely admit. The counter-intuitive lesson of §11, that a smaller round chosen well beats admitting everyone, is a corollary, because dropping the exposed singletons raises the round's k_{eff} (§14) while keeping every remaining member above the floor.

23 A game-based definition of behavioral anonymity

A privacy claim is only as good as its definition, so we state riverrun's guarantee as a game between a challenger and an adversary, in the style cryptography uses for indistinguishability. This makes "the actor is hidden" falsifiable.

The behavioral-anonymity game. Fix a round with an anonymity set of k members whose secrets are $s_1, \ldots, s_k$, and a public action a.

1. The challenger picks two members i_0, i_1 that the adversary names, computes both of their valid executions of a (commitment, nullifier, proof), and flips a fair coin b.
2. The challenger publishes the execution of member i_b: the public transcript $\{R, a, n\}$ and the proof.
3. The adversary, who sees the entire public ledger, the funding graph, and every other member's public data, outputs a guess b'.

The adversary's **advantage** is $\left|\Pr[b' = b] - \frac{1}{2}\right|$. Behavioral anonymity holds if this advantage is negligible.

Claim 23.1 (What riverrun proves, and against which adversary)**.** Against an adversary restricted to the on-chain transcript and proof, riverrun's advantage bound is negligible: the transcript $\{R, a, n\}$ is a root shared by all members, a public action identical for both, and a nullifier that is a PRF output unlinkable to any commitment (§16), and the proof transmits no witness bits (§7.3). So on the protocol surface the two executions are indistinguishable. Against an adversary that *additionally* reads the funding graph, the advantage is bounded by the provenance structure: if i_0 and i_1 lie in the same provenance class, the advantage stays negligible, and if they lie in different classes, the adversary wins, which is exactly the statement that the real anonymity is k_{eff}, not k (§14), and exactly why the ruler measures it and the coordinator forms same-class rounds.

The definition earns its keep by being honest about the second adversary. A paper that proved only the first bound, indistinguishability on the protocol surface, would be true and misleading, because the funding-graph adversary is real and cheap. riverrun's claim is the conjunction: indistinguishable on-chain, and measured against provenance, with the measurement reported rather than assumed. That is the whole thesis in one game.

24 Formal notions of privacy: from computational to everlasting

"Post-quantum" and "everlasting" are used loosely in this field, so we fix the ladder of definitions and place riverrun on it. Consider a scheme that hides a secret s behind a public value v (a pseudonym, a commitment, a proof), and an adversary trying to learn s or to link two values to the same s. The hiding can be of four strengths.

Computational. No polynomial-time adversary can distinguish, but an unbounded one, or a future faster one, can. Security rests on a problem being hard *now*. Deployed discrete-log pseudonyms are here (§5.1): safe against today's machines, broken by a quantum one, retroactively.

Statistical. The distributions the adversary sees are statistically close, distinguishable only with negligible advantage *even by an unbounded adversary*. No amount of computation helps, because the information is nearly absent, not merely scrambled.

Perfect. The distributions are identical. The adversary's view is independent of s; there is literally nothing to extract. Shamir's scheme below the threshold (§16) is perfect: N shares of a degree-N polynomial are distributed identically for every possible secret.

Everlasting. A hybrid tuned to the on-chain threat: the transcript recorded today leaks nothing about s even to a future unbounded or quantum adversary, though the protocol may rely on a computational assumption *during* execution that need only hold for the brief moment of the interaction, not forever. This is the right notion for a permanent ledger, where the recorded data is what an adversary harvests, and the computation to break it happens later.

Claim 24.1 (Where riverrun sits)**.** riverrun ID's per-context unlinkability is everlasting. The recorded values, the pseudonym Rescue(s, θ), the commitment, the nullifier, are hash outputs, and linking two of them or inverting one to s reduces to preimage or collision resistance, against which the only quantum attack is Grover's bounded speedup (§15). No recorded value reduces to a problem Shor breaks, so the transcript on the permanent ledger cannot be unwound by a future quantum adversary. The RLN secret below its threshold is stronger still, perfectly hidden. What is *not* claimed is formal zero-knowledge of the STARK proof (§18): the proof keeps the witness off the wire but is not proven to leak nothing, so its hiding is asserted empirically, not by a simulator, and that gap is named rather than folded into "everlasting."

The ladder clarifies the strong claim and its honest boundary in one place. The *unlinkability* of the identities, the property a user relies on for years, is everlasting. The *zero-knowledge* of one proof, a property that matters only to whoever holds that proof off-chain, is the weaker "witness not on the wire." Confusing the two would let a paper claim more than it has; separating them is the discipline the whole document runs on.

25 Adversary models and their costs

"Secure" is meaningless without an adversary, and "an adversary" is too coarse. A privacy tool faces a spectrum, each rung cheaper for the attacker and stronger against the defender, and honesty means saying which rungs riverrun stops and at what cost to whom.

The passive observer. Reads the public ledger, nothing more. Cost: near zero. Sees the transcript $\{R, a, n\}$ and the proof. riverrun defeats this adversary outright: the transcript is shared across the set and the proof carries no witness (§23), so the actor is one of k with equal likelihood.

The funding-graph analyst. Additionally traces provenance and keeps a tag database of exchange and service addresses. Cost: low, and the going rate of a commercial chain-analysis subscription. This is the adversary that collapses k to k_{eff}, and the one riverrun does not defeat but *measures*: the ruler reports the real crowd against exactly this attacker, the coordinator forms same-provenance rounds to shrink its advantage, and the preflight warns a user before they act. The honest posture is that this adversary is real and cheap, so we quantify its success rather than assume it away.

The behavioral cluster analyst. Additionally fingerprints timing, sizing, and sequence. Cost: moderate, requiring models and history. The synchronized round (§8) is the defense: identical actions at one moment erase timing and ordering, driving this adversary's attribution to chance, measured at 12.5% at k=8 and 1.5% at k=64, i.e. $1/k$.

The colluding committee. A quorum of the settlement committee, in the pre-M31 deployment, could attest to a false proof. Cost: corrupting M of N distinct signers. riverrun does not eliminate this adversary in the deployed program; it *prices* it at M collusions rather than one, and names its elimination, the on-chain M31 verifier, as the roadmap (§13).

The future quantum adversary. Harvests the chain today, breaks it later. Cost: a cryptographically-relevant quantum computer, plus free storage now. riverrun defeats this adversary by construction on everything recorded (§15), which is the entire argument for being hash-only, and the one property no curve-based competitor can retrofit.

The pattern across the rungs is deliberate. Where an adversary is cheap and unavoidable, the funding-graph analyst, riverrun measures it and defends where it can rather than pretending it is absent. Where an adversary is expensive or future, the quantum one, riverrun defeats it outright because the cost of doing so, at design time, is zero. Where an adversary sits on a trusted party we have not yet removed, the committee, riverrun prices the trust and publishes the plan to price it to zero. A threat model that claimed to stop all five equally would be the dishonest kind. The value is in saying which is which.

26 The economics of the anonymity set

§2 argued that anonymity is a collective good made of other people. That has consequences an engineer must design for, because a shared resource with the wrong incentives is depleted. This section reads the anonymity set as an economic object.

The set is a commons with a congestion externality of an unusual sign. In most commons, each additional user degrades the resource (a crowded pasture, a congested road). An anonymity set is the opposite: each honest additional member *improves* the good for everyone, because a larger crowd hides each of its members better. The externality is positive, which means the private incentive to join is weaker than the social value of joining, the classic under-provision of a public good. A rational user waits for others to build the crowd before entering, and if everyone reasons so, the crowd never forms.

Two failures follow, and riverrun addresses each by construction. The first is the *free-rider who depletes*: a member who acts many times consumes anonymity, acting a thousand times turns a crowd of a thousand into one person in a thousand masks (§6). The nullifier and the RLN construction (§16) are the enclosure that prevents this, capping each member's consumption without a central registry. The second is the *coordination failure*: even honest members must act at the same

time, from the same provenance class, for the crowd to be real, and no individual can arrange that alone. The synchronized round and the coordinator (§11, §22) supply the missing coordinator, forming the crowd on purpose rather than hoping it self-assembles.

> The structural isomorphism, stated and bounded. A privacy pool's participation problem is a **stag hunt**: everyone is better off if all cooperate to form a large crowd, but each fears being the lone early mover whose action stands out, so the safe individual choice is to defect and wait. *Disanalogy:* in the classic stag hunt the payoff is fixed once players choose, whereas here the coordinator changes the game by refusing to fire a round until the crowd is safe, converting "act now and risk exposure" into "act when $k_{\text{eff}} \geq k_{\min}$ is guaranteed." It removes the incentive to defect by removing the exposure that defection avoided.

Anti-Sybil is priced, not solved, and the economics say why that is hard. An adversary can always manufacture more identities by funding more wallets, so the funding graph is both the leak the ruler measures and the cost the attacker pays. riverrun's entry fee raises the price of inflating a set without claiming to make it infinite (§28), and the honest reference for the stronger construction, one-nullifier-per-verifier self-credentials, is named rather than implied. The economic point is that behavioral anonymity cannot be bought to perfection at zero cost by either side: the defender pays in coordination and fees, the attacker pays in funding and identities, and the ruler is the instrument that makes both prices visible.

27 The threat model, plainly

A privacy claim without a threat model is not falsifiable, and an unfalsifiable claim is not an engineering claim. So here is the adversary RIVERRUN is built against.

What the adversary can do. See the entire public ledger forever, every transaction, amount, address, and the funding graph behind any wallet, run modern chain-analysis, and keep a tag database of exchange and service addresses. Observe the pool's public record $\{R, a, n\}$ for every execution. For the funding-graph results, also observe the provenance class of the *acting* identity, which is cheap because a fresh withdrawal wallet has to be funded from somewhere visible.

What RIVERRUN defends. The link between an actor and their action. Given an execution, the adversary should do no better than $1/k_{\text{eff}}$ at naming the author, where k_{eff} is the effective size of that member's provenance class, and the tools here *measure* k_{eff} rather than assume it.

What RIVERRUN does not defend, by design. Amounts and balances, which are public here (hiding them is a separate, composable problem). The value layer of consensus: the ledger records that value moved, and no circularity changes that. The circularity lives on the attribution layer an analyst reads, not the settlement layer consensus enforces. And, in the deployed program today, the correctness of the membership proof, delegated to the verifier committee until on-chain M31 verification lands on mainnet. Every number in this paper is stated against this adversary.

28 What does not hold yet

A paper that only lists its strengths is an advertisement. In one place, what RIVERRUN does not yet do:

- **The circuit is hand-rolled and unaudited.** Mutation testing removes worthless tests. It is not an audit.
- **It is not formally zero-knowledge.** The prover does not randomize the witness, so the honest claim is "the secret is not on the wire", checked empirically.
- **On-chain verification is proven in a local VM, not on mainnet.** The M31 relation verifies at 159,849 CU in LiteSVM. The deployed program still leans on the M-of-N committee until that ships to mainnet.
- **riverrun ID's in-circuit membership is not built.** The seven-power primitive is tested. The Poseidon2 Merkle proof over M31 and its Solana verifier are the road ahead.
- **Security parameters are example-grade** in the Winterfell path, roughly 84 bits, not the 128 a deployment needs.
- **"Rootless" provenance is conditional.** The circularity holds only while a construction's funding sources are themselves unattributable.
- **The ricorso proof is specified, not built**, for the set-size alignment reason of §9.
- **Anti-Sybil is priced, not prevented.** Money still buys k, though the funding graph makes the cheap version of the attack visible to the ruler.

The implementation is Rust, MIT-licensed, with 154 passing tests across the core, the STARK, the pool, the ruler, and the on-chain program, and clean static analysis. Every measurement is reproducible from the repository with one command.

29 Comparison to prior systems

riverrun is a synthesis, and the honest way to show it is to place it beside the systems it draws from and say, dimension by dimension, where it is behind and where it is different. The table below compares along five axes that this paper has argued matter: *what is hidden* (value or actor), the *strength* of the unlinkability (computational or everlasting), whether it is *post-quantum*, whether it needs a *trusted setup*, and whether it *measures* the anonymity it delivers.

system	hides	unlinkability	post-quantum	trusted setup	measures k_{eff}
Tornado Cash	value	computational	no (curves)	yes (ceremony)	no
Zcash (Sapling)	value + parties	computational	no (curves)	yes (ceremony)	no
Monero	value + parties	computational	no (curves)	no	no
Aztec	value + state	computational	no (curves)	yes	no
Semaphore (Eth)	actor (identity)	computational	no (curves)	yes	no
RLN	actor, rate-limited	computational	no (curves)	yes	no
riverrun	**actor (behavior)**	**everlasting**	**yes (hash)**	**no**	**yes**

Read the table without triumphalism. On *maturity and proof size*, riverrun is behind every deployed system in the list: Zcash and Tornado are battle-tested at scale with years of adversarial attention,

and pairing-based systems produce proofs two orders of magnitude smaller than a STARK. What the last three columns show is not that riverrun is better everywhere, but that it occupies a corner none of the others do: it is the only one whose unlinkability is everlasting rather than computational, the only one that is post-quantum by construction, and the only one that measures the real anonymity it delivers rather than advertising a count. The first two properties come from the single design choice of being hash-only; the third is the ruler.

The nearest neighbors are instructive. **Semaphore** is riverrun ID's closest analogue, a secret identity with scoped nullifiers giving unlinkable per-context identity and one-action-per-person, and riverrun ID does not claim to invent that shape. The differences are exactly the table's last three columns: Semaphore's unlinkability rests on a group under discrete log and is therefore computational and not post-quantum, it uses a Groth16 backend with a trusted setup, and it does not measure anonymity-set quality. riverrun ID is Semaphore's shape rebuilt on hashes, on a chain that has no Semaphore, with a ruler attached. **RLN** is the rate-limiting construction riverrun's sixth power adopts wholesale (§16), with the same discrete-log-to-hash substitution. And **Tornado** is the pool's structural template, a Merkle set with a zero-knowledge withdrawal, with the payload changed from a coin to a behavior and the proof system changed from a ceremony-bound SNARK to a transparent STARK. The synthesis is real, and so is the debt.

30 Open problems and directions

The honest list of what does not hold (§28) is about the current implementation. This section is about the harder questions the design opens, stated as problems rather than promises, because a research contribution is measured partly by the questions it makes precise.

In-circuit membership over a small field. The identity-bearing powers (turn, grant, credential) and the pool's membership proof need a Poseidon2 Merkle path arithmetized as an AIR over M31 and verified on-chain (§19, §21). The hash foundation on vetted parameters is built; the arithmetization and its no_std verifier are the concrete next milestone, and their completion is what turns riverrun ID from a tested primitive into a deployed layer.

A repeated-use bound that is tight, not just sound. The erosion bound of §14 uses basic composition, a worst-case floor. A tight bound would model the correlation structure of real persistent quasi-identifiers, funding origin, timing, device, and give a per-identity privacy budget that is accurate rather than conservative. This is the "measured toward provable" direction: the metric exists, its optimality does not, and closing that gap is a paper in its own right.

Formal zero-knowledge without abandoning transparency. The witness is off the wire but not proven to leak nothing (§18). A transparent, post-quantum, formally zero-knowledge STARK for riverrun's relation, via witness masking that survives the small-field setting, would remove the one asterisk on the proof's privacy. The reference lines exist; a drop-in does not.

Provable-privacy anchors. Beyond measurement lies the goal of a privacy guarantee reducible to a worst-case hardness assumption, for instance a cover construction whose indistinguishability is as hard as a lattice problem, turning "measured k_{eff}" into "provably k_{eff} against any polynomial adversary." This is speculative and named as such, the most distant item on the list.

Distributed proving for crowds of strangers. The batched round assumes one prover holding all secrets (§8), which fits an operator running many identities but not a crowd of mutually distrustful users. A multi-party proving protocol would let strangers form a batched round without a custodian,

extending the efficiency gain from fleets to genuine crowds.

Each of these is a place where riverrun is honestly incomplete, and naming them precisely is the point. A tool that measures the gap between advertised and delivered privacy owes the same honesty about the gap between what it is and what it could be.

31 Related work

RIVERRUN's *goal*, hiding which member of a set acted, is anonymous authentication, a mature field. On Ethereum the identity layer is **Semaphore**, and riverrun ID is a post-quantum, Solana-native cousin. The everlasting-versus-computational unlinkability question is live in the anonymous-credential world: deployed BBS pseudonyms are only computationally unlinkable [9], while riverrun ID's hash pseudonyms are everlasting, of the secret-free quantum-safe class Kazmi and Minwalla built [8]. PLUME [7] formalizes the nullifier; PrivDID [10] states the session-unlinkability goal; anonymous-credential frameworks with predicate proofs [11, 12] share the threat model. The *metric* is Serjantov and Danezis [1], whose own note on repeated use [2] motivates the erosion bound of §5.2. The *programme* of measuring advertised-versus-true anonymity is established on Ethereum [3, 4, 5, 6]. On-chain STARK verification on Solana has been done, both with Winterfell [13, 18] and with the M31 Circle-STARK verifier [17] we build on. What is narrow and new here is the combination: the payload is behavior rather than value, one secret becomes an everlastingly-unlinkable per-context identity on a chain with no Semaphore, and the funding graph is *measured* rather than assumed away, then turned into a user-facing pre-flight defense and a crowd-forming agent with checkable certificates.

32 Conclusion

Solana was missing an anonymity layer, and RIVERRUN is one, built post-quantum from the first line. Its core is an identity: one secret, a different unlinkable persona in every app, accountable where it matters, and unlinkable in a way a quantum computer cannot undo. On top of it sits a pool that hides a behavior instead of a coin, proven by a transparent STARK whose on-chain verification we have now shown inside the Solana VM at 159,849 compute units. And a ruler keeps the whole thing honest, on Solana mainnet today, reporting its own reliability so a rate-limited network never reads as "private."

The honest version of a privacy system ships its own strongest attack and reports what it finds. That is what we tried to build. We attacked our own settlement and closed three findings with tests. We measured the gap between the privacy a set advertises and the privacy it delivers, and turned that measurement into a defense.

The cypherpunks were right that no one grants you privacy. You build it or you go without. What that generation could not yet do was *prove* to you that what they built worked. RIVERRUN is one attempt to close that gap: privacy you do not have to take on faith, on a chain that will still be public and permanent long after the machine that threatens it exists. Build it post-quantum, ship your own strongest attack, and let the reader check. Then the claim is not "trust us." It is "check."

A Reproducing every number

The system is open-source and MIT-licensed. Everything runs from one script, `./demo.sh`. Individually:

claim	command
154 tests green	`cargo test -workspace`
riverrun ID's seven powers	`cargo test -p riverrun-core rotatable rln`
the turn, proven in zero knowledge	`cargo test -p riverrun-stark rotation`
repeated-use erosion of an identity	`cargo test -p riverrun-trace repeated`
one execution's cost	`cargo run -release -example bench`
the batched round	`cargo run -release -example behavior_pool`
the coordinator + certificate	`cargo run -release -example coordinator`
k_{eff} on RIVERRUN's designs	`riverrun exhibit`
k_{eff} of a live pool	`riverrun audit <POOL> 30`
your anonymity before you act	`riverrun preflight <WALLET>`
M31 verification inside the Solana VM	`cargo test -p ...stark-verifier -test cu`

The live-data commands read only public chain data, name no individual, and print their own bounds: a clean result is a floor, "no cheap attribution found", never a proof of anonymity.

References

[1] A. Serjantov, G. Danezis. *Towards an Information Theoretic Metric for Anonymity.* PET 2002 (Outstanding Paper). https://bib.mixnetworks.org/pdf/serjantov2002towards.pdf

[2] G. Danezis. *Measuring anonymity: a few thoughts and a differentially private bound.* http://www0.cs.ucl.ac.uk/staff/G.Danezis/papers/Danezis-MeasuringThoughts.pdf

[3] N. Wu et al. *Tutela: Assessing User-Privacy on Ethereum and Tornado Cash.* arXiv:2201.06811. https://arxiv.org/abs/2201.06811

[4] F. Béres et al. *Blockchain is Watching You.* arXiv:2005.14051. https://arxiv.org/abs/2005.14051

[5] *Clustering Deposit and Withdrawal Activity in Tornado Cash.* arXiv:2510.09433 (2025). https://arxiv.org/abs/2510.09433

[6] H. Du et al. *Breaking the Anonymity of Ethereum Mixing Services Using Graph Feature Learning.* IEEE TIFS, 2024. https://doi.org/10.1109/TIFS.2023.3326984

[7] A. Gupta, K. Gurkan. *PLUME: An ECDSA Nullifier Scheme.* IACR ePrint 2022/1255. https://eprint.iacr.org/2022/1255

[8] R. A. Kazmi, C. Minwalla. *Anonymous Credentials: Secret-Free and Quantum-Safe.* Bank of Canada SWP 2023-50. https://www.banqueducanada.ca/wp-content/uploads/2023/09/swp2023-50.pdf

[9] *A Brief Note on Cryptographic Pseudonyms for Anonymous Credentials.* arXiv:2510.05419 (2025). https://arxiv.org/abs/2510.05419

[10] *Toward Verifiable Privacy in Decentralized Identity (PrivDID).* IACR ePrint 2026/127. https://eprint.iacr.org/2026/127

[11] *Formalizing Privacy of Anonymous Credentials: A Framework with Predicate Proofs.* IACR ePrint 2026/1373. https://eprint.iacr.org/2026/1373

[12] *Re2creds: Reusable Anonymous Credentials.* IACR ePrint 2026/119. https://eprint.iacr.org/2026/119

[13] J. Yano. *Full L1 On-Chain ZK-STARK+PQC Verification on Solana: A Measurement Study.* IACR ePrint 2025/1741. https://eprint.iacr.org/2025/1741

[14] A. Szepieniec, T. Ashur, S. Dhooghe. *Rescue-Prime: a Standard Specification (SoK).* IACR ePrint 2020/1143. https://eprint.iacr.org/2020/1143

[15] Facebook/Meta. *Winterfell: a STARK prover and verifier.* https://github.com/facebook/winterfell

[16] StarkWare. *Stwo: a Circle STARK prover over M31.* https://github.com/starkware-libs/stwo

[17] *Murkl: a Circle STARK verifier as a Solana CPI target.* https://github.com/exidz/murkl

[18] Wiener Labs. *Mosaic: on-chain verifier library for Solana (chunked FRI-STARK).* https://github.com/wienerlabs/mosaic

[19] C. E. Shannon. *A Mathematical Theory of Communication.* Bell System Technical Journal, 1948.

[20] C. Diaz, S. Seys, J. Claessens, B. Preneel. *Towards Measuring Anonymity.* PET 2002.

[21] C. Dwork, A. Roth. *The Algorithmic Foundations of Differential Privacy.* Found. and Trends in Theoretical Computer Science, 2014.

[22] S. Warren, L. Brandeis. *The Right to Privacy.* Harvard Law Review, 1890.

[23] A. Westin. *Privacy and Freedom.* Atheneum, 1967.

[24] R. Gavison. *Privacy and the Limits of Law.* Yale Law Journal, 1980.

[25] J. Bentham. *Panopticon; or, the Inspection-House.* 1791.

[26] M. Foucault. *Discipline and Punish: The Birth of the Prison.* 1975 (Eng. trans. 1977).

[27] D. Chaum. *Untraceable Electronic Mail, Return Addresses, and Digital Pseudonyms.* Communications of the ACM, 1981.

[28] D. Chaum. *Security without Identification: Transaction Systems to Make Big Brother Obsolete.* Communications of the ACM, 1985.

[29] T. May. *The Crypto Anarchist Manifesto.* 1988.

[30] E. Hughes. *A Cypherpunk's Manifesto.* 1993. https://www.activism.net/cypherpunk/manifesto.html

[31] *Bernstein v. United States* (9th Cir.), establishing source code as protected expression, 1990s.

[32] P. W. Shor. *Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer.* SIAM J. Computing, 1997 (FOCS 1994).

[33] L. K. Grover. *A Fast Quantum Mechanical Algorithm for Database Search.* STOC 1996.

[34] M. Mosca. *Cybersecurity in an Era with Quantum Computers: Will We Be Ready?* IEEE Security & Privacy, 2018.

[35] A. Shamir. *How to Share a Secret.* Communications of the ACM, 1979.

[36] E. Ben-Sasson, I. Bentov, Y. Horesh, M. Riabzev. *Fast Reed-Solomon Interactive Oracle Proofs of Proximity (FRI).* ICALP 2018. And: *Scalable, transparent, and post-quantum secure computational integrity (STARK).* IACR ePrint 2018/046.

[37] Rate-Limiting Nullifier (RLN). Logos Research. https://research.logos.co/rln

[38] D. J. Solove. *'I've Got Nothing to Hide' and Other Misunderstandings of Privacy.* San Diego Law Review, 2007.

[39] H. Nissenbaum. *Privacy as Contextual Integrity.* Washington Law Review, 2004; and *Privacy in Context*, Stanford University Press, 2010.

[40] S. Zuboff. *The Age of Surveillance Capitalism.* PublicAffairs, 2019.

[41] P. Zimmermann. *PGP (Pretty Good Privacy).* 1991.

[42] W. Dai. *b-money.* 1998. http://www.weidai.com/bmoney.txt

[43] N. Szabo. *Bit Gold.* c. 1998.

[44] D. Das, S. Meiser, E. Mohammadi, A. Kate. *Anonymity Trilemma: Strong Anonymity, Low Bandwidth Overhead, Low Latency, Choose Two.* IEEE S&P 2018.